

Area and Performance Tradeoffs in Floating-Point Divide and Square-Root Implementations

PETER SODERQUIST

Cornell University, Ithaca

AND

MIRIAM LEESER

Northeastern University, Boston

Floating-point divide and square-root operations are essential to many scientific and engineering applications, and are required in all computer systems that support the IEEE floating-point standard. Yet many current microprocessors provide only weak support for these operations. The latency and throughput of division are typically far inferior to those of floating-point addition and multiplication, and square-root performance is often even lower. This article argues the case for high-performance division and square root. It also explains the algorithms and implementations of the primary techniques, subtractive and multiplicative methods, employed in microprocessor floating-point units with their associated area/performance tradeoffs. Case studies of representative floating-point unit configurations are presented, supported by simulation results using a carefully selected benchmark, Givens rotation, to show the dynamic performance impact of the various implementation alternatives. The topology of the implementation is found to be an important performance factor. Multiplicative algorithms, such as the Newton-Raphson method and Goldschmidt's algorithm, can achieve low latencies. However, these implementations serialize multiply, divide, and square root operations through a single pipeline, which can lead to low throughput. While this hardware sharing yields low size requirements for baseline implementations, lower-latency versions require many times more area. For these reasons, multiplicative implementations are best suited to cases where subtractive methods are precluded by area constraints, and modest performance on divide and square root operations is tolerable. Subtractive algorithms, exemplified by radix-4 SRT and radix-16 SRT, can be made to execute in parallel with other floating-point operations. Combined with their reasonable area requirements, this gives these implementations a favorable balance of performance and area across different floating-point unit configurations. Recent developments in microprocessor technology, such as decoupled superscalar implementations and increasing instruction issue rates, also favor the parallel, independent operation afforded by subtractive methods.

Categories and Subject Descriptors: B.2.1 [**Arithmetic and Logic Structures**]: Design Styles—*parallel, pipeline*; C.4 [**Computer Systems Organization**]: Performance of Systems—*design studies*; C.5.3 [**Computer System Implementation**]: Microcomputers—*microprocessors*; G.1.0 [**Numerical Analysis**]: General—*computer arithmetic*

General Terms: Algorithms, Design, Performance

Additional Key Words and Phrases: Area and performance tradeoffs, division, floating-point, FPU, square root, SRT

This research is supported in part by the National Science Foundation under contracts CCR-9257280. Peter Soderquist was supported in part by a Fellowship from the National Science Foundation. Miriam Leeson is supported in part by an NSF Young Investigator Award.

Authors' Addresses: P. Soderquist, School of Electrical Engineering, Cornell University, Ithaca, NY 14853; email: pgs@cs.cornell.edu; M. Leeson, Dept. of Electrical and Computer Engineering, Northeastern University, Boston, MA 02115; email: mel@ece.neu.edu.

Permission to make digital/hard copy of part or all of this work for personal or classroom use is granted without fee provided that the copies are not made or distributed for profit or commercial advantage, the copyright notice, the title of the publication, and its date appear, and notice is given that copying is by permission of the ACM, Inc. To copy otherwise, to republish, to post on servers, or to redistribute to lists, requires prior specific permission and/or a fee.

© 1996 ACM 0360-0300/96/0900-0518 \$03.50

1. INTRODUCTION

Floating-point divide and square-root operations occur in many scientific and commercial computer applications. Any system complying with the IEEE 754 floating-point standard, including the majority of current microprocessors, must support these operations accurately and correctly. Yet very few implementations have divide and square-root performance comparable to that of the other basic mathematical operations, addition and multiplication. In many cases, floating-point division and square root are implemented with a minimum of hardware resources, resulting in vastly inferior relative performance. Furthermore, within a given arithmetic unit, there is typically a significant difference between division and square root themselves. Performance also varies widely across different processors, even among designs competing directly in the same overall price/performance class. Most designers appear willing to sacrifice speed in favor of lower cost and design complexity. This policy originates in the misconception that division and square root are relatively insignificant operations. We argue, through concrete examples, that division and square root are critical to real applications, and that their performance and area requirements should be balanced with those of other arithmetic operations.

There are two major categories of divide and square-root algorithms, *multiplicative* and *subtractive* methods, and within each category a considerable number of design variables. Designers are sometimes intimidated by what can be a complex, bewildering subject, and the temptation to opt for the easiest possible solution is great. This article explores and clarifies the range of practical divide and square-root alternatives, explaining the algorithms and their implementations. More importantly, it provides an analysis of the cost and performance tradeoffs associated with different implementations,

enabling designers confronted with these choices to make as well-informed decisions as possible. The discussion applies primarily to general-purpose microprocessors supporting the IEEE 754 floating-point standard. In addition, the focus is on double-precision operations, which predominate in science and engineering applications.

1.1 Related Work

Other researchers have performed comparisons of different divide and square-root implementations. The earliest studies are relatively selective, usually focusing on a small subset of the possible alternatives. In his seminal paper on higher-radix divide algorithms, Atkins [1968] considers how to compute the cost of different types of SRT implementations. Stearns [1989] discusses the merits and demerits of multiplicative and subtractive algorithms and presents a design for SRT division and square root which incorporates “the best of the ideas presented over the last thirty years” along with a few innovations of his own. While quickly dismissing multiplicative methods, Peng et al. [1987] give a detailed but largely qualitative discussion of the area and performance properties of a variety of subtractive divide methods. Taylor [1985] performs a systematic study of ten related but different SRT divider designs with a quantitative comparison of cost and performance. Ercegovic and Lang [1994] present a time and area analysis for a smaller but more diverse group of subtractive dividers; Ercegovic et al. [1993] perform a similar study of very high radix divide methods. These studies focus almost exclusively on subtractive methods, without a substantive comparison with multiplicative techniques.

More recent investigations of the subject have been more inclusive. Oberman and Flynn [1994b] analyze the system-level impact of several divide implementation options, including parallelism, divide latency, and instruction issue rate,

using simulations of the SPEC benchmark suite. Later work by the same authors [1994a] features an extensive survey of divide algorithms and implementations, with discussions of area requirements and per-operation performance but without performance simulation. These studies focus almost entirely on division to the exclusion of square root, and leave aside many FPU-level tradeoffs and micro-architectural issues.

In contrast with similar research by other parties, our work is primarily concerned with the problems of implementing divide and square-root operations within the context of the *floating-point unit (FPU)*. Division and square root are considered together since the algorithms are similar and support efficient unified implementations. Significant interactions with other FPU components and operations are also thoroughly explored. Instead of focusing on per-operation performance, or investigating a small design space with a set of generic benchmarks, we use the simulation of a single, carefully chosen benchmark to explore a wide, diverse range of practical design alternatives. This benchmark, Givens rotation, is a significant, real application which illuminates the performance impact of divide/square-root implementations in a readily quantifiable way. Finally, we undertake all simulations at the FPU level rather than the system level, to investigate the tradeoffs independently of compiler effects and other non-floating-point concerns.

1.2 Overview

The remainder of this article has the following structure. Section 2 explains why divide and square-root implementations are worthy of serious consideration. Section 3 provides an overview of floating-point unit configurations and the role of divide and square root operations within them. Section 4 explains multiplicative algorithms and implementations, while Section 5 does the

same for subtractive methods. Section 6 focuses on the major area and performance tradeoffs inherent in different implementation options. Section 7 extends the performance analysis using simulations of the Givens rotation benchmark on a set of representative floating-point configurations. Finally, Section 8 provides concluding remarks.

2. THE IMPORTANCE OF DIVISION AND SQUARE ROOT

Floating-point division and square root are operations whose significance is often downplayed or accorded minimal weight. The purpose of this section is to argue that divide and square-root implementation should be granted a more prominent role in the FPU design process. The case of the Pentium division bug and its expensive consequences for Intel illustrates some of the dangers of implementing division with insufficient care (Appendix A). The more insidious problem, however, is the gap between division and square-root performance and that of other operations. First, a look at current microprocessors demonstrates the disparity in concrete terms. An analysis of common perceptions reveals the reasons for this state of affairs, followed by a discussion of the consequences and arguments in favor of a different policy.

2.1 Performance of Current Microprocessors

Early microprocessors had meager hardware support for floating-point computation. These chips implemented many operations, particularly divide and square root, in software, and required floating-point coprocessors for reasonable arithmetic performance. By contrast, the majority of recent general-purpose microprocessor designs, including most low-end devices, contain built-in arithmetic units with hardware support for addition, multiplication, and division at the very least.

The floating-point unit designs of re-

Table 1. Performance of Recent Microprocessor FPU's for Double-Precision Operands (* = inferred from available information; † = not supported)

Design	Cycle Time [ns]	Latency [cycles]/Throughput [cycles]			
		$a \pm b$	$a \times b$	$a \div b$	$\sqrt{a}$
DEC 21164 Alpha AXP	3.33 ns	4/1	4/1	22–60/22–60*	†
Hal Sparc64	6.49 ns	4/1	4/1	8–9/7–8	†
HP PA7200	7.14 ns	2/1	2/1	15/15	15/15
HP PA8000	5 ns	3/1	3/1	31/31	31/31
IBM RS/6000 POWER2	13.99 ns	2/1	2/1	16–19/15–18*	25/24*
Intel Pentium	6.02 ns	3/1	3/2	39/39	70/70
Intel Pentium Pro	7.52 ns	3/1	5/2	30*/30*	53*/53*
MIPS R4400	4 ns	4/3	8/4	36/35	112/112
MIPS R8000	13.33 ns	4/1	4/1	20/17	23/20
MIPS R10000	3.64 ns	2/1	2/1	18/18	32/32
PowerPC 604	10 ns	3/1	3/1	31/31	†
PowerPC 620	7.5 ns	3/1	3/1	18/18	22/22
Sun SuperSPARC	16.67 ns	3/1	3/1	9/7	12/10
Sun UltraSPARC	4 ns	3/1	3/1	22/22	22/22

cent microprocessors reveal the perceived importance of addition and multiplication. Most FPU's are clearly optimized for these operations, while support for division and square root ranges from generous to minimal. Consider some performance figures from recent microprocessors, shown in Table 1. All figures are for double precision operands. Addition latency ranges from 2 to 4 machine cycles, multiplication between 2 and 8. The majority of processors have latencies of 2 or 3 machine cycles for both operations. By contrast, division latencies range from 8 to 60 machine cycles. Square root performance for hardware implementations covers an even wider range, from 12 to 112 machine cycles; processors which implement the operation in software generally take even longer.

Throughput performance is even more biased in favor of addition and multiplication. Contemporary FPU's universally feature heavily pipelined addition and multiplication hardware, yielding repeat rates of 1 or 2 cycles in most cases. Hardware specifically for division and square root is primarily nonpipelined, which means that the repeat rate is typically within one or two cycles of the latency. Furthermore, in some processors, executing a divide or square root

instruction prevents other floating-point operations from initiating until the computation is complete.

As the examples illustrate, while division is slow on many processors, square root is often significantly slower. It is difficult to justify why this should be the case. In most instances, it is possible to implement square root in conjunction with division, with performance as good as or only slightly worse, and at a relatively low marginal cost.

2.2 Perceptions

Clearly, there is no common standard for the performance of divide and square root operations. Performance drastically inferior addition and multiplication also seems perfectly acceptable. The reasons for this policy lie in a mixture of clear fact and mere perception. Most of the design effort and transistor budget for FPU's goes into the addition and multiplication pipelines and supporting components such as the floating-point register file. If functionality needs to be scaled back due to area or other constraints, square root and division are the first and second targets, respectively, for simplification or outright elimination. This leads to the widely varying levels of performance in

different machines. Low divide and square root performance are considered an acceptable sacrifice because designers regard these operations as relatively unimportant. This evaluation stems from their apparent infrequency in the instruction stream, which minimizes the perceived consequences of poor performance. Insuring the efficiency of addition and multiplication, however, is paramount.

Multiplication and addition/subtraction are unquestionably the most common arithmetic operations. The methods which chip designers use to evaluate instruction frequencies tend to amplify this perception. Code traces from the real applications of intended users give the most accurate indication of workloads. But assembling a balanced sample of application code is difficult, tedious, and fraught with uncertainty. Designers, therefore, tend to rely on benchmarks for insight into instruction frequencies. Performance evaluation experts have made a convincing case that so-called synthetic and kernel benchmarks, like Whetstone and Linpack, respectively, are not representative of typical floating-point workloads [Dixit 1991; Hennessy and David 1990; Weicker 1991]. The SPEC [Case 1995; Dixit 1991] and Perfect Club [Berry et al. 1988] benchmarking suites are more recent attempts to provide useful and meaningful metrics. Both employ programs used in real applications, in their entirety, rather than short, artificial programs or disembodied code fragments. Yet Whetstone, Linpack, and other benchmarks like them are still used ubiquitously to compare machines ranging from supercomputers [Berry et al. 1991] to low-end desktop workstations [Juliussen 1994], and therefore continue to affect design criteria.

There is also a vicious cycle at work. Computers have traditionally been poor at performing divide and square root operations, originally because the design of efficient implementations was poorly understood [Scott 1985]. As a consequence, numerical analysts have

favorable algorithms which avoid these operations over equivalent methods which use them extensively [McQuillan and McCanny 1991]. For example, one motivation for using the so-called “fast” or “square root free” Givens rotation algorithm instead of the standard one is the fact that it has no explicit square root operations [Golub and Van Loan 1989]. Even relatively recent work on adaptive signal processing advertises “square root and division free” algorithms [Frantzeskakis and Liu 1994]. In light of this, computer designers examining actual end users’ code might conclude that division and square root are indeed relatively unimportant operations, thus perpetuating the tradition of weak implementations.

2.3 Realities

The trend favoring poor divide and square root implementations has unfortunate side effects. Many of the algorithms derived to avoid division and/or square root display poor behavior, such as numerical instability or a tendency to overflow. In that regard, they are inferior to the original formulations [Kahan 1994; McQuillan and McCanny 1991]. The fast Givens rotation, for example, suffers from the risk of overflow, while the standard Givens rotation does not. The fact is, many algorithms are most naturally formulated in terms of division and square root. Providing adequate support for these operations is a feasible, desirable alternative to convoluted programming tricks.

Pipelining and increased design sophistication have provided marked improvements in the latency and throughput of addition and multiplication. Divide and square root implementations have generally not kept pace, which means that these operations have increasingly become bottlenecks and determinants of overall performance in applications which dare to use them at all. Compiler optimization only aggravates the situation by reducing excess instructions and increasing the relative

frequency of divide and square root operations [Oberman and Flynn 1994a]. The performance mismatch is especially hard on those implementations which not only have high latency but cause the rest of the floating-point unit to stall while computing quotients or roots. Even in processors with independent divide/square root units, excessive latencies can deplete the set of dispatchable instructions before the operation terminates, causing pipeline stalls. In short division and square root can affect FPU performance well out of proportion to their frequency of occurrence [McQuillan et al. 1993; Scott 1985]. Any assessment of implementation costs should hold this fact in consideration.

Contrary to conventional wisdom, there are common, important, and non-trivial applications where divide and/or square root efficiency make a critical difference in overall performance. One such algorithm, employed in a wide range of scientific and engineering applications, is Givens rotation, which is explored further in later sections. One survey of floating-point programs, including typical code from SPICE simulations, found a typical divide to multiply instruction ratio of 1:3 or higher [Peng et al. 1987]. A proposed superscalar architecture, designed for optimal execution of the SPEC89 benchmark set, calls for fully pipelined dividers with 10 cycle latencies, as compared with 3 cycles for multiplication [Conte 1993]. Another study using the SPECfp92 benchmark suite found that while floating-point division comprised only 3% of the dynamic floating-point instruction count, given a 20 cycle latency, the operations would account for 40% of overall program latency [Oberman and Flynn 1994b]. The specific numbers and criteria used are open to debate, but one thing is clear: divide and square root performance are important to floating-point performance in general and should not be short-changed.

3. IMPLEMENTING FLOATING-POINT ARITHMETIC

In a microprocessor, division and square root must contend for time and space with other floating-point operations, particularly addition and multiplication. These competing operations must be smoothly integrated into a structure which implements the required functionality. General-purpose microprocessors usually group arithmetic circuitry into a dedicated floating-point unit and provide software functions for features not supported in hardware. This section examines the properties of floating-point units relevant to their role as divide and square root environments.

3.1 Divide and Square Root Algorithms

There are many different methods for computing quotients and roots, although relatively few have seen practical application, especially in microprocessor FPU's. The divide and square root algorithms in the current machines fall into the two categories of subtractive and multiplicative techniques, named for the principal step-by-step operation in each class [Wong and Flynn 1992].

Multiplicative Methods. Multiplicative algorithms, represented by the *Newton-Raphson method* and *Goldschmidt's algorithm*, do not calculate quotients and square roots directly, but use iteration to refine an approximation to the desired result. In essence, division and square root are reduced to a series of multiplications, hence the name. The rate of convergence is typically quadratic, providing for very high performance in theory. Implementations can also re-use existing hardware, primarily the floating-point multiplier present in all FPU's. In recent years, multiplicative techniques have declined in popularity. Subtractive methods provide competitive latency, and re-use of the floating-point multiplier for division and square root can save area but risks creating a performance bottleneck.

There are, however, several current designs which utilize multiplicative algorithms, such as the IBM RS/6000 POWER2 [Markstein 1990; Misra 1990; White et al. 1993; White 1994] and MIPS R8000 [Mips Technologies 1994b]. Section 4 is devoted to the theory and implementation of multiplicative division and square root.

Subtractive Methods. Subtractive algorithms calculate quotients directly, digit by digit. The so-called “pencil-and-paper” method for long division taught in elementary school is a member of this class; there is an analogous technique for the subtractive computation of square roots. As the name implies, subtraction is the central operation of these algorithms. The many variations of SRT division are examples of this type. Years of research have produced increasingly sophisticated, efficient, and complex variants of these basic methods, which have become the most popular means of performing division and square root in the latest microprocessors. For example, 11 out of the 14 chips in Table 1 perform division using subtractive hardware, and all but two of those do the same for square root. Subtractive implementations can achieve low latencies, and have a relatively small hardware cost. This means that divide and square root computation can be readily provided in parallel with other operations, potentially improving FPU throughput. Section 5 discusses subtractive algorithms and implementations in detail.

3.2 Floating-Point Unit Configurations

The organization of a floating-point unit is largely determined by the implementation of addition and multiplication, but there are still several degrees of freedom. Figure 1 displays some of the most common floating-point unit topologies. All of the diagrams assume a dedicated floating-point register file and issue/retirement rates of one operation per cycle. The first structure, shown in Figure 1(a), is referred to as a *chained*

configuration and is usually associated with area-constrained implementations. The multiplier is generally a partial array requiring multiple passes for double-precision operands. To save even more hardware, the adder performs the final carry-propagate addition and rounding for the multiplier and divide/square root unit. In the configuration of Figure 1(b), addition is independent, and divide and square root computation are dependent on multiplication. Figure 1(c) shows the most elaborate topology. Division and square root are coupled, but the other operations are completely independent and execute in parallel. The last configuration, in Figure 1(d), has a considerably different design philosophy from the others and is based on a multiply-accumulate structure. Its primary operation is the atomic multiplication and addition of three operands. Internal routing, registers, and tables extend the multiply-accumulate pipeline for divide and square root functionality; all operations are performed in series.

The FPU's in current microprocessors are all related to one of the configurations in Figure 1; the Mips R4400 FPU looks like Figure 1(a), while the SuperSPARC resembles Figure 1(b). The HP PA7100 FPU is like Figure 1(c), except that it can issue and retire a multiply and either an add or divide/square root operation concurrently. Figure 1(d) is based on the IBM RS/6000 series floating-point units. The HP PA8000 is a cross between the configurations of Figure 1(c) and (d), combining a multiply-accumulator with an independent divide/square root unit.

Notice the consistent pairing of division and square root. For a given class of algorithm there is usually a strong similarity between the methods of computing division and square root, and considerable opportunities for hardware sharing. Where hardware resources are devoted to improving divide performance, designers often exploit the opportunity to incorporate square root

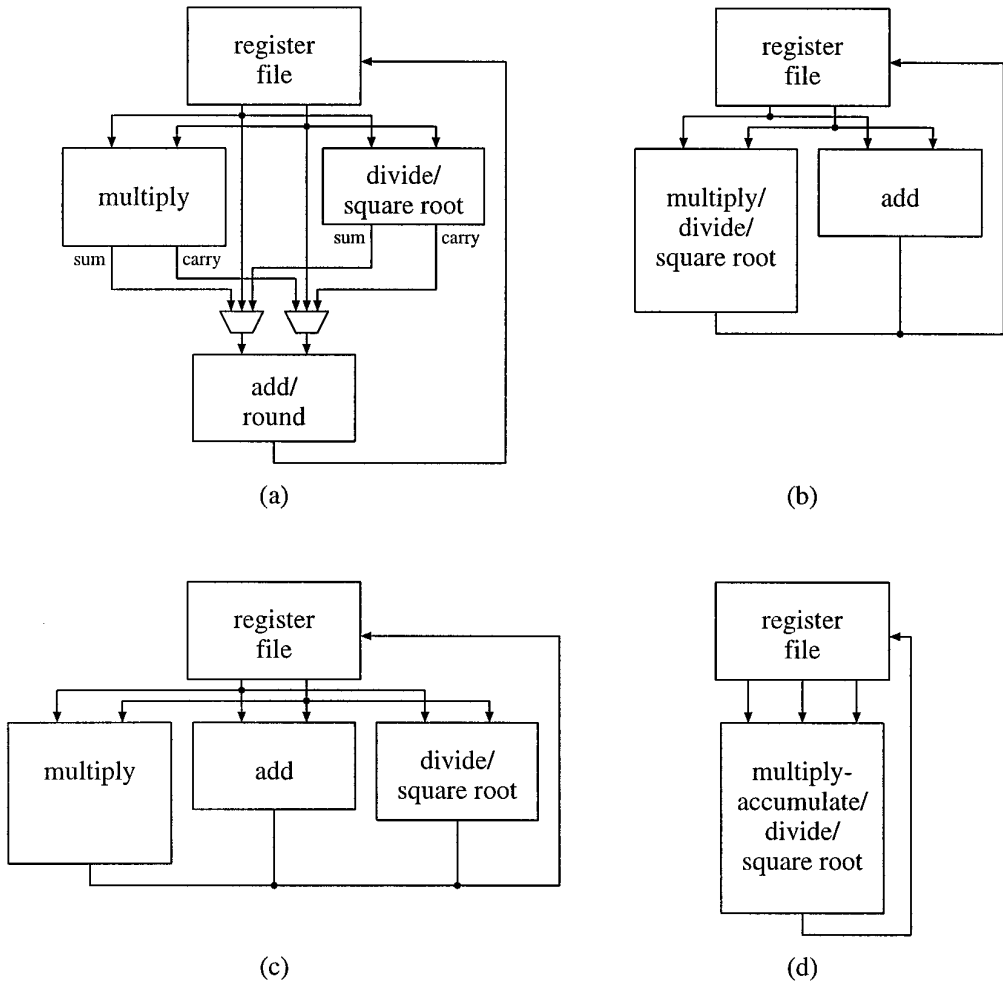


Figure 1. Common floating-point unit configurations.

functionality as well—although they occasionally delegate it to software.

Lest the reader forget, floating-point values consist of three components: sign, fraction, and mantissa. This article focuses almost exclusively on the manipulation of fractional values. While correct processing of signs and exponents is absolutely essential, the implementation problems are comparatively trivial. Another important subject not thoroughly explored in this text is the correct handling of floating-point exceptions, such as division by zero. This is a thorny, highly machine-specific topic which extends far beyond the scope of

this article. A good reference for insights into the fundamental issues is the IEEE 754 floating-point standard [IEEE 1985].

4. MULTIPLICATIVE METHODS

Although currently less popular than their subtractive counterparts, multiplicative algorithms are utilized in several contemporary microprocessors, and remain a feasible alternative in some applications. This section explains the theory and practice of multiplicative division and square root computation. There are two different but related mul-

multiplicative techniques in current use, the Newton-Raphson method and Goldschmidt's algorithm, described in the first part of this section. The similarity of these methods leads to similar hardware implementations, which are the subject of the latter part.

4.1 Algorithms

The primary appeal of multiplicative methods, also known as *functional iteration* methods, is their potential for very low latencies. Multiplicative methods use iterative approximation to converge on a result, typically at a quadratic rate. This means that the number of result digits is effectively doubled at each iteration. By contrast, subtractive methods add the same number of bits to the result at each step, giving a linear rate of convergence. Of course, asymptotic convergence and actual performance are two different things, but multiplicative divide and square root implementations typically yield lower latencies than subtractive ones of comparable complexity.

Out of the possible multiplicative algorithms, two have been adopted in recent microprocessor and arithmetic coprocessor designs. The Newton-Raphson method has its roots in the 17th century and has been widely used for years in both hand-held calculators and general-purpose computers, including the current IBM RS/6000 series [Markstein 1990]. Goldschmidt's algorithm has been employed to a lesser extent, first in the IBM System/360 Model 91 mainframe [Anderson et al. 1967], but most notably in recent years by Texas Instruments in arithmetic coprocessors and some implementations of the SuperSPARC architecture [Darley et al. 1990; Hennessy and Patterson 1990b; Sun Microsystems Computer Corp 1992].

The Newton-Raphson Method. The Newton-Raphson method [Hennessy and Patterson 1990b; Scott 1985] works by successively approximating the root of an equation. Given a continuous func-

tion $f(x)$ with a root at X , and an initial guess $x_0 \approx X$, the Newton-Raphson method yields a recurrence on x , where successive values of x_i are increasingly closer to X . To perform the division a/b with the Newton-Raphson method, let $f(x) = 1/x - b$. This equation has a root at $x = 1/b$. If $0 < x_0 < 2/b$, where x_0 is the initial guess or *seed value*, the Newton-Raphson iteration

$$x_{i+1} = x_i \times (2 - b \times x_i) \quad (1)$$

converges on this root to the desired accuracy. Multiplying by a yields an arbitrarily precise approximation to a/b .

Square root computation is similar and also based on a reciprocal relationship. To compute $\sqrt{a}$, let $f(x) = 1/x^2 - a$, which has a root at $x = 1/\sqrt{a}$. With an initial guess $0 < x_0 < \sqrt{3/a}$, iteration over

$$x_{i+1} = \frac{1}{2} \times x_i \times (3 - a \times x_i^2). \quad (2)$$

followed by multiplication by a produces the desired result $\sqrt{a}$.

The Newton-Raphson divide and square root algorithms are quite similar in form. Both require a fixed number of multiplications (two for division, three for square root) and a subtraction from a constant at each step, followed by a final multiplication. The square root iteration also requires a division by 2, a trivial one-bit shift.

There is a relationship between the accuracy of the seed and the execution time of the algorithm. The Newton-Raphson iteration has a quadratic rate of asymptotic convergence, which means that the precision of the estimated result approximately doubles at each step. As a consequence, the number of iterations required to achieve double precision accuracy is coupled to the accuracy of the initial guess.

Unfortunately, there is a numerical pitfall associated with multiplicative algorithms like Newton-Raphson which use iterative refinement. If only nominal precision is maintained throughout

the computation, then the result may deviate from the IEEE standard result in the two least significant bits [Hennessy and Patterson 1990b]. Solutions to this problem are covered in the discussion of implementations.

Goldschmidt's Algorithm. Goldschmidt's algorithm is derived from a Taylor series expansion [Scott 1985]; it is mathematically related to the Newton-Raphson method and has similar numerical properties, including the last-digit accuracy problems. Let a be the dividend and b the divisor. Computing the quotient x_0/y_0 ($x_0 = a$, $y_0 = b$) with Goldschmidt's algorithm involves multiplying both the numerator and denominator by a value r_i such that $x_{i+1} = x_i \times r_i$ and $y_{i+1} = y_i \times r_i$. Successive values of r_i are chosen such that $y_i \rightarrow 1$, and therefore $x_i \rightarrow a/b$; the selection is implemented as $r_i = 2 - y_i$. To insure rapid convergence, both numerator and denominator are prescaled by a seed value close to $1/b$.

Square root calculation is similar. To find $\sqrt{a}$, let $x_0 = y_0 = a$ and iterate over $x_{i+1} = x_i \times r_i^2$ and $y_{i+1} = y_i \times r_i$ so $x_{i+1}/y_{i+1}^2 = x_i/y_i^2 = 1/a$. Choose successive r_i 's such that $x_i \rightarrow 1$, and then consequently $y_i \rightarrow \sqrt{a}$. Values of r_i are obtained through the formula $r_i = (3 - y_i)/2$, and the prescaling operation uses an estimate of $1/\sqrt{a}$.

Note that the type and number of operations performed in each iteration are the same for both Goldschmidt's algorithm and the Newton-Raphson method, even though the order is different. Both techniques also have quadratic convergence. While Goldschmidt's algorithm avoids the final multiplication required by the Newton-Raphson method, the prescaling operations take the same amount of time as one iteration. It would appear that the two methods have equivalent performance, with Newton-Raphson having a slight edge. However, Goldschmidt's algorithm has the advantage that the numerator and denominator multiplications are independent operations. This provides for significantly more

efficient utilization of pipelined multiplier hardware than the Newton-Raphson method, where each step depends on the result of the previous one.

4.2 Implementations

Most of the following discussion applies equally to the Newton-Raphson method and Goldschmidt's algorithm, which have similarities beyond their use of multiplication. Where reference to a particular algorithm is required, Goldschmidt's algorithm is used because of its higher performance potential.

Software. Although all microprocessors of recent design perform division in hardware, some use software to implement square root, including the DEC Alpha 21164 [Bannon and Keller 1995], some members of the PowerPC family [Becker et al. 1993; Burgess et al. 1994; Song et al. 1994], and the original IBM RS/6000 [Markstein 1990]. Because of their quadratic convergence, multiplicative algorithms are the method of choice for software implementations; the major stumbling block is the problem of last-digit accuracy. Obtaining properly rounded results requires access to more bits of the final result than n , the precision of the rounded significand [Markstein 1990], something not all architectures readily provide at the instruction level. The time expense of obtaining a correctly rounded result in software can be high. The Intel i860 provides reciprocals and root reciprocals rounded to nominal precision in hardware, using the Newton-Raphson method, with no additional precision for the final multiplication. Cleaning up the last two bits in software takes as much time as finding the initial estimate [Hennessy and Patterson 1990b], even though the instruction set provides access to bits in the lower product word [Kohn and Margolis 1989].

Hardware. Multiplicative hardware virtually always consists of modifications to existing functional units. The block diagram in Figure 2 shows a two-

stage pipelined floating-point multiplier modified for computing division and square root. Dashed lines indicate components and routing paths not needed for multiplication alone. This particular implementation is geared towards Goldschmidt's algorithm, but a Newton-Raphson version would have similar elements. The most noteworthy features are the extra routing and temporary storage, the lookup table for seed values, and a unit designed to provide the subtraction from a constant and the shift required by the iteration.

Routing and Control Modifications. The first step to achieving speedup over software implementations is to extend the floating-point controller to perform the multistep iterative formulas atomically as single instructions. This entails modifying the control logic, and providing bypass routing and registers (if necessary) so that intermediate values can be fed repeatedly through the multiplier. The extra routing removes both the necessity and delay of accessing the register file for intermediate computation steps. This eliminates possible contention with the floating-point adder and prevents blocking. One other optimization is possible in multipliers with partial arrays, where double precision values must cycle through the array several times. In such units, one can exploit the quadratic convergence of multiplicative algorithms and perform the early iterations in reduced precision, since only the final iteration requires values with full-width accuracy.

Rounding and Accuracy Considerations. Correct rounding of the final result requires better than n -bit precision for accuracy to the last bit. This can be difficult to achieve in software, since the floating-point instructions on many architectures only return values rounded to standard precision. In hardware, it is relatively simple to increase precision across multiple operations merely by extending the width of the datapath. Since wider datapaths are more expensive, one would like to know

the minimum precision required to achieve the desired accuracy. One old rule of thumb holds that the reciprocal and numerator product should both be computed to $2n$ bits prior to rounding to insure correctness in the last digit [Hennessy and Patterson 1990b]. This is the approach taken by the IBM RS/6000 floating-point unit, which consists of an atomic multiply-accumulate circuit. The entire $2n$ -bit product is summed with the addend to $3n$ bits of precision [Montoye et al. 1990]. IBM scientists have proven that the Newton-Raphson implementation of the RS/6000 generates correctly rounded quotients and square roots [Markstein 1990]. Some designers, daunted by the prospect of double-width datapaths and longer convergence times, have opted to trade accuracy for higher performance and lower costs, as with the previously-cited Intel i860. The slightly more elaborate reciprocal approximation scheme of the Astronautics ZS-1 is fully accurate most of the time, but still differs from the IEEE specification in a small number of cases [Fowler and Smith 1989].

The Texas Instruments TMS390C602A [Darley et al. 1990] and 8847 [Hennessy and Patterson 1990b] arithmetic coprocessors demonstrate that full divide and square root accuracy can be achieved without a large hardware expenditure or performance sacrifice. The rounding scheme is discussed in more detail by Darley et al. [1989] and applies to both chips. The TMS390C602A and 8847 multipliers have 60-bit datapaths, with space for a 53-bit double precision significand, the guard, round, and sticky bits, and four extra guard digits. For division, the quotient $q = a/b$ is computed to extra precision. Then $q \times b$ is computed, also to 60 bits of precision, and compared with a in the lowest order bits to find the direction of rounding for q . For square roots, a tentative root $r = \sqrt{a}$ is computed. Then $r \times r$, the square of the approximate root, is compared with the radicand a , all with extra precision, and r is rounded accordingly.

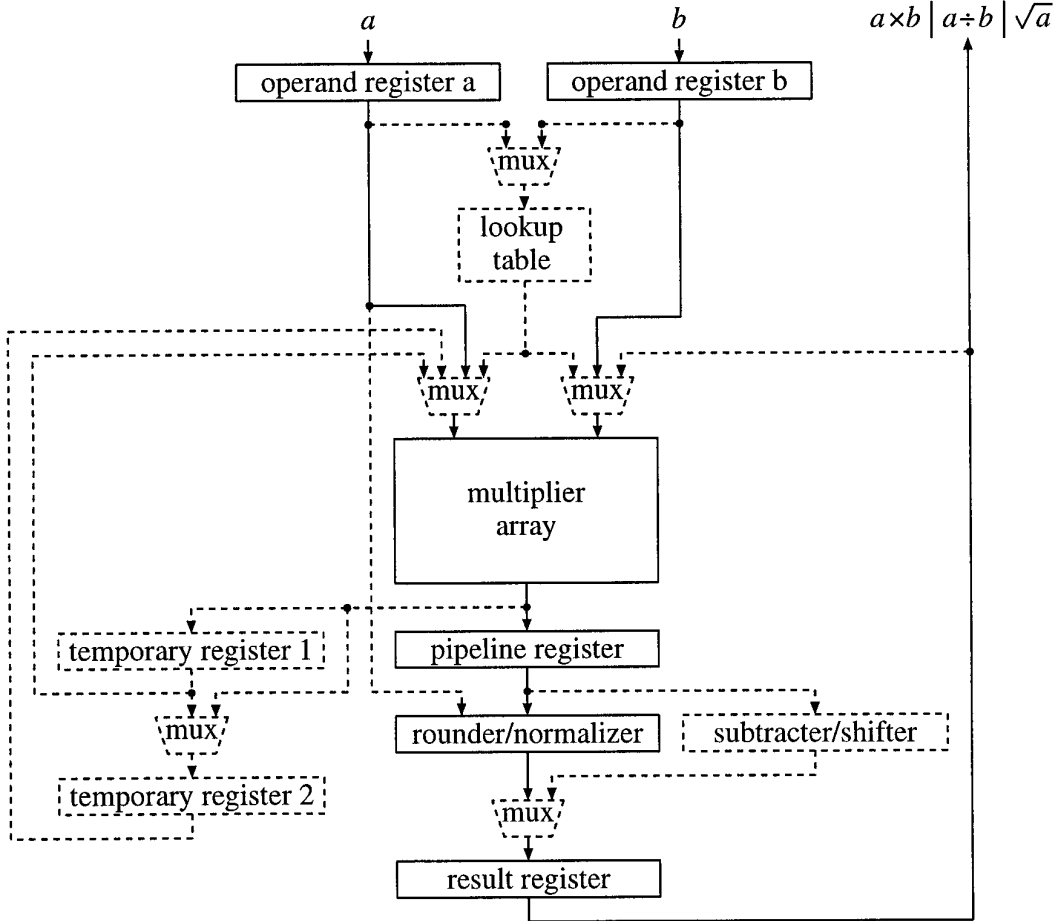


Figure 2. An independent floating-point multiplier enhanced for high divide/square root performance.

This procedure yields double precision results correctly rounded to the last bit. The comparison operation affects only the last few bits of the result and the input operand, so the hardware cost and performance expense of a full-width comparison is not necessary. Kabuo et al. [1994] offer another rounding technique requiring a 60-bit datapath and a cleanup time on the order of a single multiplication. The implementation, however, is both more complicated and constraining, being tightly coupled to the design of the floating-point multiplier itself.

Lookup Tables. The single most valuable enhancement to the perfor-

mance of multiplicative hardware is a lookup table providing the initial guess at the reciprocal value. Because of the quadratic convergence of multiplicative algorithms, using a table can give a valuable head start and drastically reduce the number of iterations required to achieve the desired accuracy. For example, if the accuracy of the initial guess is one bit, it will take six iterations to reach an accuracy of 64 bits. With an initial guess accurate to 8 bits, the number of iterations required drops to just three. The use of lookup tables is a standard feature of multiplicative implementations like the IBM RS/6000 series.

A reciprocal table takes the k bits after

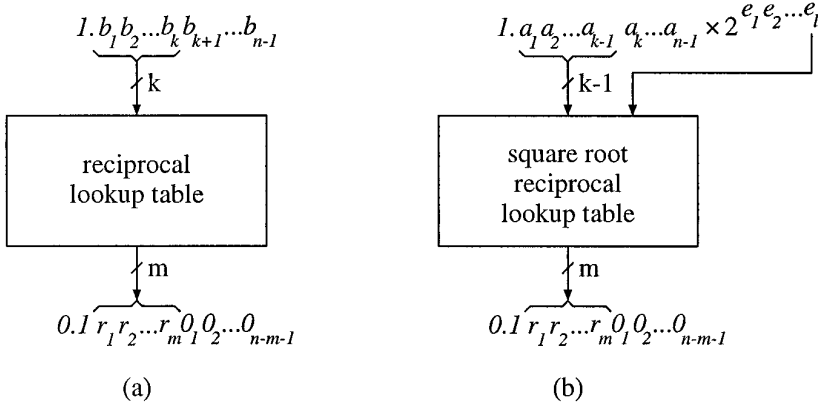


Figure 3. Input-output connections for k -bits-in, m -bits-out (a) reciprocal and (b) square root reciprocal lookup tables.

the binary point of the input and returns an m -bit guess, where $m \geq k$. Consider finding the reciprocal of a normalized n -bit number $b = 1.b_1b_2\dots b_{n-1}$. The lookup table uses $b_1b_2\dots b_k$ as an index and returns the n -bit value $0.1r_1r_2\dots r_m0_2\dots 0_{n-m-1}$ where $0.1r_1r_2\dots r_{n-1} = r \approx 1/b$. This process is illustrated in Figure 3. Recall that all normalized IEEE 754 values have a leading one, and that this value does not need to be read into the table; likewise, there is no need for the table to store the leading 1 or $n - m - 1$ trailing zeros.

A square root reciprocal table works in a similar way, but with a twist. A k -bits-in, m -bits-out table is indexed by the $k - 1$ first bits of the fraction and the last bit of the exponent, as shown in Figure 3. The reciprocal of b and the reciprocal of $b/2$ differ by a factor of two, a mere binary shift reflected in the exponent field with no effect on the mantissas. By contrast, the root reciprocals of a and $a/2$ differ by a factor of $\sqrt{2}$, so the proper initial guess for a given input will depend on whether its exponent is even or odd. An alternative method is to require that operands have either all odd or all even exponents, shifting them as needed prior to lookup on the fraction. In this case, one cannot assume a normalized fraction.

Iterative Step Support. Recall that the Newton-Raphson and Goldschmidt

iterations for division and square root are series of multiplications interspersed with subtractions and shifts. Performing these auxiliary operations in the multiplier instead of accessing the floating-point adder not only speeds up computation but preserves the independence of the functional units. This can simplify instruction scheduling and improve FPU throughput. Furthermore, fixed-point subtraction from constants can be implemented with far simpler hardware than a generalized floating-point subtraction, and supporting a one-bit shift is also trivial. The implementation in Figure 2 has a separate unit devoted to these operations. Another possibility is to extend the multiplier array with extra routing and another row of adders, and perform the constant subtractions in conjunction with the multiplication [Lu and Dawallu 1989]. A variation of this scheme executes the subtractions and shifts in the circuits which recode the multiplier operands into redundant form for passage through the array [Kabuo et al. 1994]. The availability of signed digits makes this easy to do without incurring significant area or performance penalties. The details of these implementations are tied to the design of the multiplier.

5. SUBTRACTIVE METHODS

Although once regarded as slow, excessively complicated to implement, and generally impractical, subtractive methods have grown increasingly popular, facilitated by a deepening understanding of the algorithms and progressively more efficient implementations. The subtractive methods covered in this section are often grouped under the heading of *SRT* divide/square root. *SRT* stands for D. Sweeny, J.E. Robertson, and K.D. Tocher, who more or less simultaneously developed division procedures using very similar techniques [Scott 1985]. This section contains an overview of subtractive algorithms, followed by a description of implementation techniques.

5.1 Algorithms

Subtractive methods compute quotients and square roots directly, one digit at a time; for this reason, they are also known as *digit recurrence* algorithms. The paper-and-pencil method of long division for decimal numbers is just one technique in this class of algorithms. The discussion in this section is much more general, applying to operands with a variety of radices and digit sets. All operands are assumed to be fractional, normalized, and precise to n bits. In the case of IEEE double-precision values, let $n = 53$, and let all significands be shifted to the right by one bit position so that the leading 1 is just to the right of the binary point. The development assumes that all operands are positive. Further details may be found in Ercegovac and Lang [1994].

The subtractive algorithms for square root are very similar to those for division. Much of the theory applies to both, and in practice, the two operations frequently have most of their hardware in common. Division will be discussed first, since it is the most familiar and simplest of the two operations, and therefore serves as the best medium for introducing the principal ideas behind the subtractive algorithms. Square root

computation will be covered later with an emphasis on those features which differ from the division algorithm.

Subtractive Division. Division is defined by the expression

$$x = q \times d + \text{rem}$$

where

$$|\text{rem}| < |d| \times \text{ulp} \quad \text{and} \quad \text{sign}(\text{rem})$$

$$= \text{sign}(x).$$

The *dividend* x and *divisor* d are the input operands. The *quotient* q and, optionally, the *remainder* rem are the results. The *Unit in the Least Position*, or *ulp*, defines the precision of the quotient, where $\text{ulp} = r^{-n}$ for n -digit, radix- r fractional results. Subtractive division computes quotients using a recurrence, where each step of the recurrence yields an additional digit. The expression

$$w[j + 1] = rw[j] - dq_{j+1} \quad (3)$$

defines the division recurrence, where q_{j+1} is the $j + 1$ st quotient digit, numbered from highest to lowest order, and $q[j]$ is the partial quotient at step j (where $q[n] = q$). The value $w[j]$ is the *residual* or *partial remainder* at step j . Initially, $w[0] = x$; that is, the partial remainder is set to the value of the dividend. The requirement that the final remainder be less than one *ulp* can be transformed into a bound on the residual at each step,

$$-d < w[j] < d.$$

This bound applies for all j , and therefore, the quotient digit q_{j+1} in Equation 3 must be chosen so that $w[j + 1]$ is properly bounded as well.

In order to make the discussion so far more concrete, consider an example using decimal numbers. Figure 4 shows the first several steps of a division operation where $x = 0.2562$ and $d = 0.3570$, displayed in the form of the pencil-and-paper method for long divi-

sion. The values are annotated with their corresponding labels in the recurrence relation. As required by any valid division operation, the condition $-d < w[j] < d$ is maintained at every step. The pencil-and-paper method, however, imposes another, more subtle set of restrictions. First of all, the partial remainder is always positive. In addition, not only are all of the quotient digits positive, but all are taken from the set $\{0, 1, \dots, 9\}$, constraints which do not apply in the general case as developed in this discussion.

Redundant Digit Sets. One important tool for speeding up subtractive division is redundant digit sets. In a nonredundant digit set for radix- r values, there are exactly r digits. The standard digit set for radix- r values is $\{0, 1, \dots, r-1\}$, as in the long division example of Figure 4. In a redundant digit set, the number of digits is greater than r . For quotient representation, most of these are of the form

$$q_j \in D_a = \{-a, -a+1, \dots, -1, 0, 1, \dots, a-1, a\},$$

i.e., symmetric sets of consecutive integers with maximum digit a . In order for D_a to qualify as redundant, a must satisfy the relation $a \geq \lceil r/2 \rceil$. The degree of redundancy is measured by the *redundancy factor*, defined as

$$\rho = \frac{a}{r-1}, \quad \rho > \frac{1}{2}.$$

The range restriction is a direct consequence of the lower bound on a . A digit representation with $a = \lceil r/2 \rceil$ is known as *minimally redundant*, while one with $a = r-1$ (and therefore $\rho = 1$) is called *maximally redundant*. If $a > r-1$ and $\rho > 1$, the digit set is known as *over-redundant*. Any representation where $a = (r-1)/2$ is nonredundant. Table 2 shows several possible quotient digit sets.

$$\begin{array}{rcl} & q & \\ d & \overline{).717\dots} & \\ .3570 & \overline{) 10 \times .2562} & rw[0] = rx \\ & \underline{-2.4990} & dq_1 \\ & 10 \times .0630 & rw[1] \\ & \underline{-.3570} & dq_2 \\ & 10 \times .2730 & rw[2] \\ & \underline{-2.4990} & dq_3 \\ & .2310 & w[3] \\ & \vdots & \end{array}$$

Figure 4. Decimal division example: $q = x \div d = 0.2562 \div 0.3570$.

Quotient-Digit Selection Regions. The efficient selection of correct quotient digits is a nontrivial problem which has long been the barrier to efficient subtractive implementations. Consider once again the paper-and-pencil method, where the (nonredundant) quotient digits are determined by experimentation. The product of a tentative q_{j+1} and d is compared to $rw[j]$; if $w[j+1] = rw[j] - dq_{j+1} > 0$, then $q_{j+1} + 1$ is tested, and so on. If k is the value of q_{j+1} which makes $w[j+1] < 0$, then the correct quotient digit is $k-1$.

When $r = 2$ there are only two possible choices of q_{j+1} , choosing the wrong value means that $w[j+1]$, which is negative, must be “restored” to $rw[j]$ by adding dq_{j+1} back in. This, in a nutshell, is the so-called *restoring* division algorithm. Its inefficiency has inspired the *nonrestoring* radix-2 algorithm, which allows negative residuals and has the digit set $\{-1, 1\}$; an “overdraft” on one iteration is compensated for by adding the divisor to the residual on the next iteration. Pseudocode for this algorithm is shown in Figure 5. Determining quotients by experimentation is simple enough for radix-2 values, where the right choice is immediately obvious from the sign of the residual, but it limits the rate of computation to one bit at a time. If the radix is 4, 8, or even 16, all of the testing and backtracking required can be quite time-consuming and expensive to automate.

SRT algorithms use more sophisticated,

Table 2. Digit Sets for Quotient Representation

r	a	Digit Set	ρ	Type
2	1	$\{-1, 0, 1\}$	1	maximally/minimally redundant
4	2	$\{-2, -1, 0, 1, 2\}$	1/3	minimally redundant
4	3	$\{-3, -2, \dots, 2, 3\}$	1	maximally redundant
4	4	$\{-4, -3, \dots, 3, 4\}$	4/3	over-redundant
8	4	$\{-4, -3, \dots, 3, 4\}$	4/7	minimally redundant
8	7	$\{-7, -6, \dots, 6, 7\}$	1	maximally redundant
9	4	$\{-4, -3, \dots, 3, 4\}$	1/2	non-redundant

efficient methods of quotient selection made possible by redundant digit representations. The following discussion is relatively concise; more extensive explanations may be found in Ercegovac and Lang [1994].

Recall that the quotient selection at each step is limited by bounds on the residual which are independent of the iteration index j . Let these bounds, $\underline{B}$ and $\bar{B}$, be defined as

$$j \underline{B} \leq w[j] \leq \bar{B}.$$

It can be shown that

$$\underline{B} = -\rho d \quad \text{and} \quad \bar{B} = \rho d,$$

where ρ is the degree of redundancy of digit set D_a . Let the *selection interval* of $rw[j]$ for $q_{j+1} = k$ be defined as $[L_k, U_k]$; that is, the range of values for which $w[j+1] = rw[j] - dk$ is correctly bounded. It can be shown that

$$U_k = (\rho + k)d \quad \text{and} \quad L_k = (-\rho + k)d$$

for a valid digit selection. This is known as the *containment condition*, and must hold true for any quotient-digit selection function. The other prerequisite for correct digit selection is known as the *continuity condition*, which states that every value of $rw[j]$ must lie on some selection interval. That is, it must be possible to choose some digit in D_a such that the next residual is correctly bounded. This can be expressed as

$$U_{k-1} \geq L_k.$$

In other words, the bounds for successive intervals must either coincide or overlap.

P-D diagrams, which plot the shifted

```

 $w = x;$ 
for  $j = 0$  to  $n - 1$ 
     $w = 2 \times w;$ 
    if  $w \geq 0$ 
         $w = w - d;$ 
         $q_{j+1} = 1$ 
    else
         $w = w + d;$ 
         $q_{j+1} = -1$ 
    end
end

```

Figure 5. Pseudocode for radix-2 nonrestoring division.

residual of a recurrence step against the divisor, are a useful method for visualizing selection intervals. The interval bounds U_k and L_k are shown as lines radiating from the origin with slope $\rho + k$ and $-\rho + k$, respectively. Figure 6 shows a P-D diagram for $r = 4$ and $a = 2$. The overlapping selection regions are shaded for clarity. Consider an iteration step where the value of $rw[j]$ is $3d/2$. On the P-D diagram, this represents a line in the shaded area between the lines for L_2 and U_1 , signifying that $q_{j+1} = 1$ and $q_{j+1} = 2$ are both valid, correctly bounded quotient-digit choices.

Quotient-Digit Selection Functions. Define the *quotient-digit selection function*, SEL, where

$$q_{j+1} = \text{SEL}(w[j], d),$$

such that $-d < w[j+1] = rw[j] -$

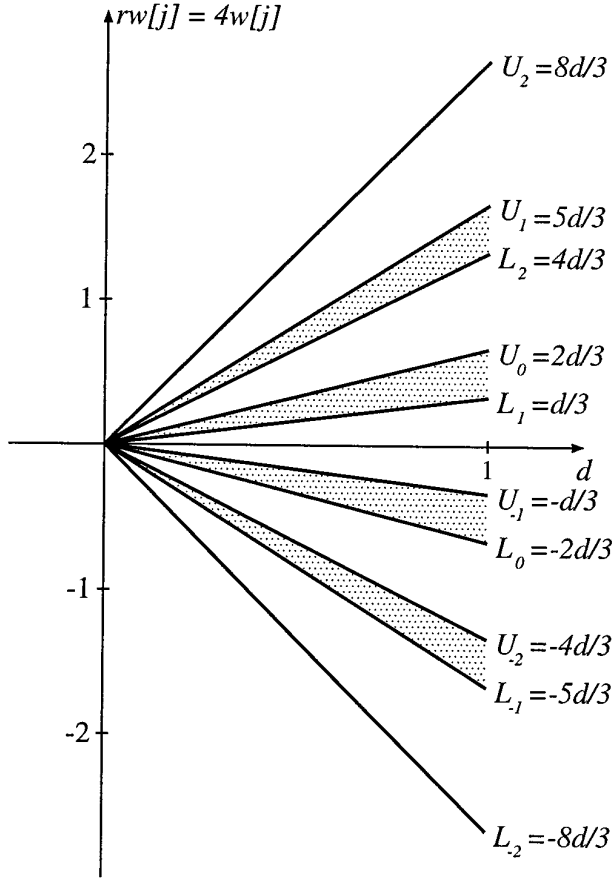


Figure 6. P-D diagram for division with $r = 4$, $a = 2$.

$dq_{j+1} < d$, that is, the residual at each iteration is correctly bounded. This function can be represented by the set of subfunctions $\{s_k\}$, $-a \leq k \leq a$, such that

$$q_{j+1} = k \quad \text{if} \quad s_k \leq rw[j] < s_{k+1},$$

with each member of $\{s_k\}$ a function of d . Obviously the s_k 's must lie on the interface between selection regions or, in the case of redundant digit sets, in the overlap of successive regions. That is,

$$L_k \leq s_k \leq U_{k-1}.$$

One of the primary motivations for using redundant quotient-digit sets is that the overlapping selection regions give

flexibility in specifying selection functions. The greater the degree of overlap, the greater the range of alternatives and opportunities for optimization. The amount of overlap is directly related to the degree of redundancy.

The most practical and commonly employed method for implementing quotient-digit selection functions uses *selection constants*. In this technique, the divisor range is split up evenly into intervals $[d_i, d_{i+1})$ where

$$d_1 = \frac{1}{2}, \quad d_{i+1} = d_i + 2^{-\delta},$$

so that the interval is specified by the δ most significant bits of the divisor. In each interval, the selection function

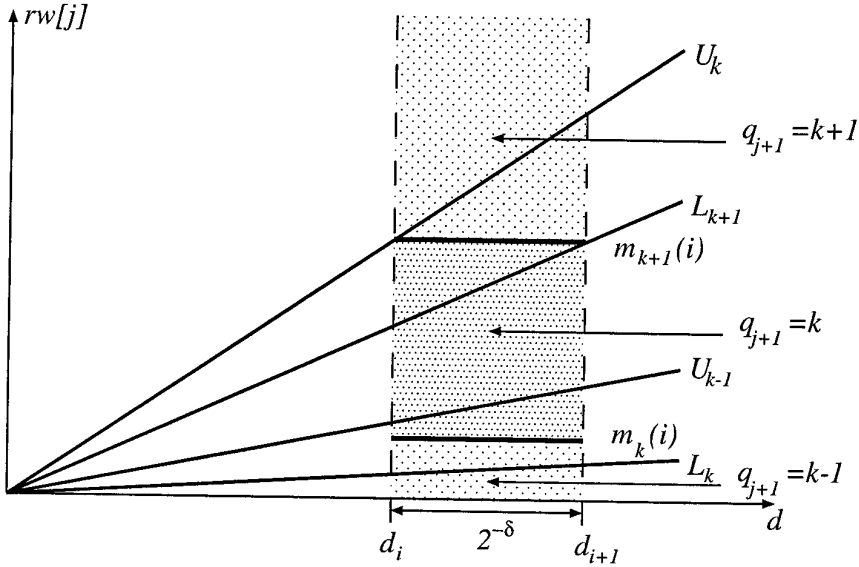


Figure 7. Selection constants for the interval $[d_i, d_{i+1})$.

contains a set of selection constants $m_k(i)$ where

for $d \in [d_i, d_{i+1})$, $q_{j+1} = k$

$$\text{if } m_k(i) \leq rw[j] \leq m_{k+1}(i) - r^{-n},$$

as shown in Figure 7. The set of selection constants for a single value of k form a “staircase” spanning the overlap between L_k and U_{k-1} , illustrated in Figure 8; every such set of constants is one member of $\{s_k\}$. It is easy to see how redundant quotient-digit representations are essential to the utility of this method.

Selection constants limit the resolution of the divisor needed for quotient selection to δ bits. Having fewer bits to work with means simpler, faster implementations of the selection function. As a further enhancement, it is possible to perform quotient selection with a truncated version of the shifted residual as well. Let $\{rw[j]\}_c$ signify the shifted partial remainder in two’s complement form truncated to c fractional bits. As will be shown below, there are performance advantages to keeping the residual in redundant form instead of two’s

complement. Actual implementations, therefore, frequently use an estimate of $\{rw[j]\}_c$ computed from the redundant representation. If y is the actual value of $rw[j]$, then let $\hat{y}$ be an estimate formed from the first t fractional bits of the redundant representation. Then $\hat{y}$ can be used as an estimate of $\{rw[j]\}_c$.

Subtractive Square Root. As mentioned earlier, the subtractive square root and division methods are closely related. This discussion likewise assumes fractional, normalized operands. The square root operation accepts a nonnegative argument x and returns the nonnegative result s , where $|x - s^2| < ulp$. The result at step j , the j th partial root, is denoted by $s[j]$. The square root recurrence is

$$\begin{aligned} w[j+1] \\ = rw[j] - 2s[j]s_{j+1} - s_{j+1}^2 r^{-(j+1)}. \end{aligned}$$

Define $f[j] = 2s[j] + s_{j+1}r^{-(j+1)}$; then $w[j+1] = rw[j] - f[j]s_{j+1}$, which is similar in form to the division recurrence. In practice, $f[j]$ is simple to generate, which facilitates combined divi-

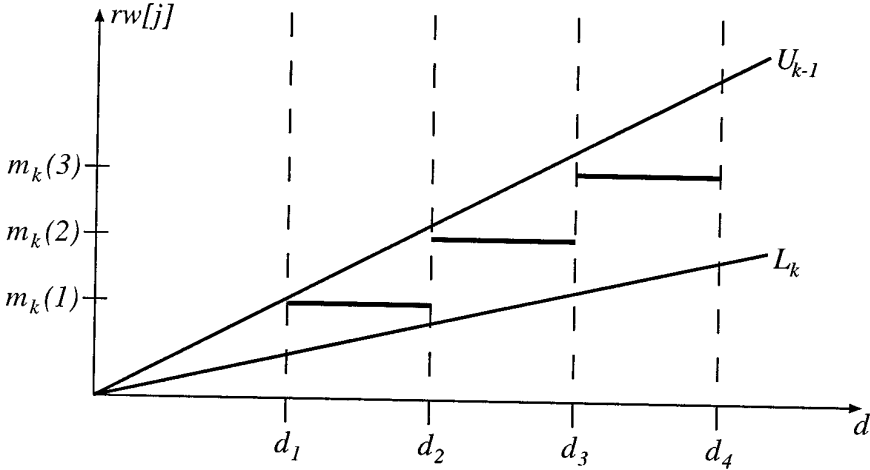


Figure 8. Selection constants for the L_k , U_{k-1} overlap region (adapted from Ercegovac and Lang [1994]).

sion and square root implementations. Result-digit sets are defined in exactly the same manner as quotient-digit sets; the same is true of the redundancy factor, ρ . These facts also assist the construction of joint divide/square root implementations.

Derivation of the square root residual bounds shows that

$$\underline{B} = -2\rho s[j] + \rho^2 r^{-j} \quad \text{and}$$

$$\bar{B} = 2\rho s[j] + \rho^2 r^{-j}.$$

Similarly, the selection interval for result digit k over the digit set with redundancy factor ρ is defined by

$$U_k = 2s[j](k + \rho) + (k + \rho)^2 r^{-(j+1)}$$

and

$$L_k = 2s[j](k - \rho) + (k - \rho)^2 r^{-(j+1)}$$

It is significant that all of the above quantities depend on the value of $s[j]$, or even j directly, which means that they vary from one iteration step to the next. In developing the quotient-digit selection function, it was noted that the residual bounds and selection intervals for division are constant over j ; no such simplifying assumption is possible for square root. It is as if there were a different P-D diagram for each itera-

tion, which complicates result-digit selection. Nevertheless, the variations can be analyzed and bounded, and with the appropriate choice of δ , t , and c , and a careful analysis of the different cases, it is possible to derive a set of selection constants which hold for all values of j . In fact, for given values of r and ρ , division and square root can generally be accommodated by the same set of constants. The various techniques described above for quotient-digit selection, including redundant, truncated residuals, can be carried over to square root result-digit selection as well. Table 3 summarizes the most important features of both the subtractive division and square root algorithms for easy reference and comparison.

5.2 Implementations

This section touches briefly on software techniques, then turns at length to hardware implementations. The basic components required for division are covered, followed by a discussion of how to combine division and square root, and finally, the more advanced subjects of on-the-fly rounding and overlapping quotient selection operation.

Software. Implementing division and/or square root operations in soft-

Table 3. Summary of Subtractive Division and Square Root Algorithm Definitions

Division	Square Root
$w[j] = r^j(x - dq[j])$	$w[j] = r^j(x - s[j]^2)$
$w[j+1] = rw[j] - dq_{j+1}$	$w[j+1] = rw[j] - 2s[j]s_{j+1} - s_{j+1}^2r^{-(j+1)}$
$\bar{B} = -\rho d < w[j] < \rho d = B$	$\bar{B} = -2\rho s[j] + \rho^2r^{-j} < w[j] < 2\rho s[j] + \rho^2r^{-j} = \bar{B}$
$\bar{U}_k = (\rho + k)d$	$\bar{U}_k[j] = 2s[j](k + \rho) + (k + \rho)^2r^{-(j+1)}$
$L_k = (-\rho + k)d$	$L_k[j] = 2s[j](k - \rho) + (k - \rho)^2r^{-(j+1)}$

ware allows one to choose from a wide variety of algorithms, unrestricted by the low-level concerns of hardware design—at least in theory. In practice, software often fails to provide adequate performance. All current microprocessors use hardware for division, and the majority provide hardware support for square root computation as well. In the mid-1980's, most microprocessors had very poor hardware floating-point support; most did not even have instructions for division or square root [Scott 1985]. Since arithmetic coprocessors were costly, some researchers recommended the use of subtractive, digit-by-digit algorithms in software. Thomas Gross [1985] proposes a subtractive division algorithm for the Mips architecture, while 1985 releases of BSD Unix contain a subtractive square root library function for the C programming language.

In more recent years, microprocessor implementations have increasingly included built in floating-point units, all of which now support addition, multiplication, and division. There are still some current designs, most notably the original IBM RS/6000 series [Misra 1990] and the Alpha AXP [McLellan 1993] which implement square root computation in software. However, both of these use multiplicative algorithms, taking advantage of their high convergence rate and the efficiency of the multiplication hardware. Subtractive software cannot compete on performance terms and has fallen out of favor.

Hardware. Subtractive division and square root are generally implemented as wholly or partially independent hardware units. While it is possible to use

the floating point adder, as is done for square roots in the Mips R4400 [Simba 1993], this is generally ill-advised, since it not only gives very poor performance but ties up one of the most frequently used functional units. More typically, subtractive methods make use of specialized logic to achieve the most favorable latencies and throughputs possible, although there is occasionally some sharing with the multiplier to reduce hardware costs.

Basic Structures. The block diagram in Figure 9 shows the structure of a basic radix-4 divider with $a = 2$, which displays the most common features of SRT implementations in general. The residual $w[j]$ (initially the dividend x) is stored in redundant form as a vector of sum bits and a vector of carry bits, while the dividend d is stored in conventional form. Multiplication of the residual by r is accomplished via a wired left shift. Quotient-digit selection takes the truncated divisor and partial remainder and produces the next quotient digit, q_{j+1} . The factor generation logic returns the product of d and q_{j+1} . The core of the divider is the carry-save adder, which performs the subtraction $rw[j] - d_{q_{j+1}}$ in each step of the iteration. In this configuration, the shifted partial remainder registers feed into the sum inputs, while the product dq_{j+1} supplies the carry bits. The result, in redundant form, feeds back to the residual registers. Finally, the on-the-fly conversion unit converts the signed-digit quotient into conventional, nonredundant form concurrently with the generation of new digits. Figure 10 gives pseudocode describing the operation of the divider.


```

ws = x;
wc = 0;
q = 0;
for j = 0 to 27
    a = 4 × ws;
    b = 4 × wc;
    qj+1 = SEL(a, b, {d}4);
    c = -qj+1 × d;
    wc, ws = CSA(a, b, c);
    q = convert(q, qj+1)
end

```

Figure 10. Pseudocode for radix-4 SRT division (**SEL** = quotient digit selection; **CSA** = carry-save addition; **convert** = on-the-fly conversion to non-redundant form).

ucts of the divisor for each member of the digit set are available “on tap” at all times. For the digit set $D_2 = \{-2, -1, 0, 1, 2\}$, this task is trivial. The factor $2d$ is a simple one-bit left shift, and along with the carry-save adder, the entire set of factors can be generated using a combination of multiplexers and inverters. Digit sets with members that are not powers of two, like $\{-3, -2, \dots, 2, 3\}$ require more hardware, including adders to create values not readily created by shifting, and possibly registers to store the generated factors.

On-the-Fly Conversion. The redundant quotient representation used internally by the divider needs to be converted back into conventional nonredundant form before being passed on to the register file or other units. Traditionally, signed-digit values have been stored as separate vectors of positive and negative digits which are then combined at the end of the computation by subtracting the negative values from the positive ones. This approach requires the presence of a full-width carry-propagate adder, and appends the delay of a full-width addition to the latency of the division operation. On-the-fly conversion computes the nonredundant representation of the result as each new digit

becomes available, with a delay comparable to that of a carry-save adder and considerably less hardware than for a carry-propagate adder [Ercegovac and Lang 1994].

The basic idea of on-the-fly conversion is to maintain two forms of the quotient: $Q[j]$, the conventional representation, and $QM[j]$, which is defined as $Q[j] - r^j$. With every new quotient digit, each of these forms is updated to its new value. It can be shown that these updates may be achieved by a combination of swapping and shifting of the old values, along with concatenation of new digits. The implementation of the on-the-fly conversion, outlined in Figure 11, has modest hardware requirements and does not add any critical path delay [Ercegovac and Lang 1994].

Combining Division and Square Root. Consider once again the similarities between the subtractive division and square root algorithms. Division is defined by the recurrence

$$w[j+1] = rw[j] - dq_{j+1},$$

while for square root

$$w[j+1] = rw[j] - f[j]s_{j+1}$$

where $f[j] = 2s[j] + s_{j+1}r^{-(j+1)}$. Combining the two operations into a single hardware unit without excessive additional area or performance penalties is predicated on two conditions. First of all, it must be possible to find a single set of selection constants which apply to both operations. It can be shown that this is the case. Typically, however, the number of bits to be examined is greater than for division alone. This is due to the dependency of the selection interval bounds on j , the iteration index [Ercegovac and Lang 1994].

The second condition is the capability to generate $f[j]$ without adding to the delay of the iteration. It turns out that this is possible with a minor extension of the on-the-fly conversion scheme. As with the quotient, two forms of the $s[j]$ are maintained, $A[j] = s[j]$ and $B[j] =$

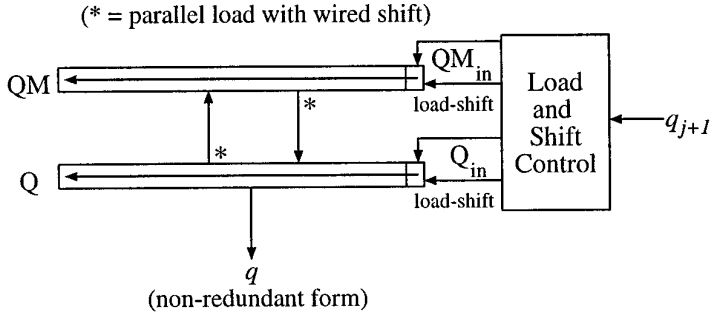


Figure 11. On-the-fly conversion implementation (adapted from Ercegovac and Lang [1994]).

$s[j] - r^{-j}$ which are analogous to $Q[j]$ and $QM[j]$, respectively. The computation of updates is similarly uncomplicated. It can be shown that the basic operations required are concatenation and shifting, which are trivial, and multiplication by a radix- r digit, the same operation required for factor generation. Because of the low latency of these operations and the ability to overlap them with the iteration step, generation $f[j]$ incurs no critical path delays. Figure 12 shows a modification of the radix-4 divider in Figure 9 which accommodates square root computation, with the structure largely unchanged. The bit-widths of the quotient digit selection have been extended to account for the variation in the selection intervals across iterations. Also, the divisor register has been combined with logic for maintaining $s[j]$ and generating $f[j]$.

Rounding. Correctly implementing round-to-nearest in compliance with IEEE 754 requires some consideration. The rounding direction of the result depends on the $(n + 1)$ st bit, q_L , and the final residual—namely, whether or not the value is nonzero, and, if so, its sign. If the residual is negative after the last iteration, then too much has been subtracted and the quotient needs to be decremented. In other cases, the quotient may have to be incremented. In order to avoid performing additions or subtractions with the associated carries and borrows, one can use yet another variation of

the on-the-fly conversion technique. In this scheme, besides the usual forms of the quotient, $Q[j]$ and $QM[j]$, a third form, $QP[j]$, is maintained, where $QP[j] = Q[j] + r^{-k}$.

The update techniques are similar to those of $Q[j]$ and $QM[j]$, requiring the same simple operations, shifting and concatenation. The resulting hardware structure is also quite similar, requiring an extra register for $QP[j]$, a slightly modified wiring scheme, and a different controller. The control logic is simple, with three input bits: q_L , and the outputs of the residual sign and zero detection logic. The circuits for sign and zero detection, on the other hand, are more complicated, requiring a carry propagation structure of the type used in full-width adders, but less costly and faster. Overall, the on-the-fly rounding scheme yields considerably less expensive hardware and higher performance than alternative methods.

Overlapping Quotient Selection. The discussion so far has used radix-4 implementations as examples, and alluded to the possibility of even higher radix implementations. A factor of 2 increase in the radix doubles the number of bits retired per iteration, giving higher radix methods obvious appeal. In theory, one could use the same basic structure presented earlier to implement division or square root of any radix r . In practice, the complexity of factor generation and quotient selection become prohibi-

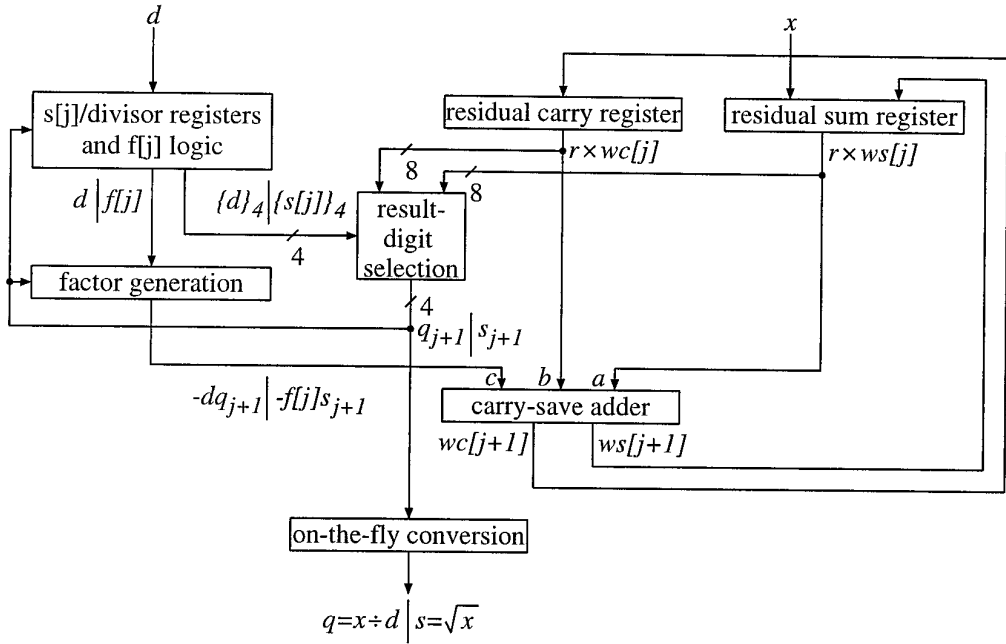


Figure 12. SRT divide/square root unit with $r = 4$, $a = 2$.

tive for $r > 8$, and the performance gain rapidly decreases. One of the most straightforward ways to achieve higher radix operations is to overlap stages with lower radices. For example, two radix-4 stages can be overlapped to obtain radix-16 division or square root.

Figure 13 demonstrates one such method for two division stages of radix r , where the quotient selection of stage $j + 1$ is overlapped with that of stage j . The method works by calculating the estimate of the residual $w[j + 1]$ and the resulting value of the next quotient digit, q_{j+2} in parallel, for all possible $2a + 1$ values of the previous quotient digit q_{j+1} . Once the actual digit becomes available, the correct value of q_{j+2} can be selected. The idea is simple and can lead to very efficient implementations, and can also be readily carried over to square root computation. In fact, this particular approach is employed in the Sun UltraSPARC, where three overlapped radix-2 stages create a radix-8 divide/square root unit. There are, however, limits to its utility. For every

stage of overlap, the number of speculative values required increases by a factor of $2a + 1$, which leads to exponential growth in circuit area.

There are even more sophisticated and complex implementations of subtractive division and square root than those described in this section. Two of the more important types are *self-timed* and *very high radix* methods. While quite rare in current microprocessor FPU's, and therefore not covered extensively in this article, they have the potential for more widespread use in future chips. Appendix B contains an overview of these techniques.

6. AREA AND PERFORMANCE TRADEOFFS

This section discusses the area and performance tradeoffs associated with the primary implementation alternatives for division and square root. Comparisons between different options are as concrete as possible given the many dif-

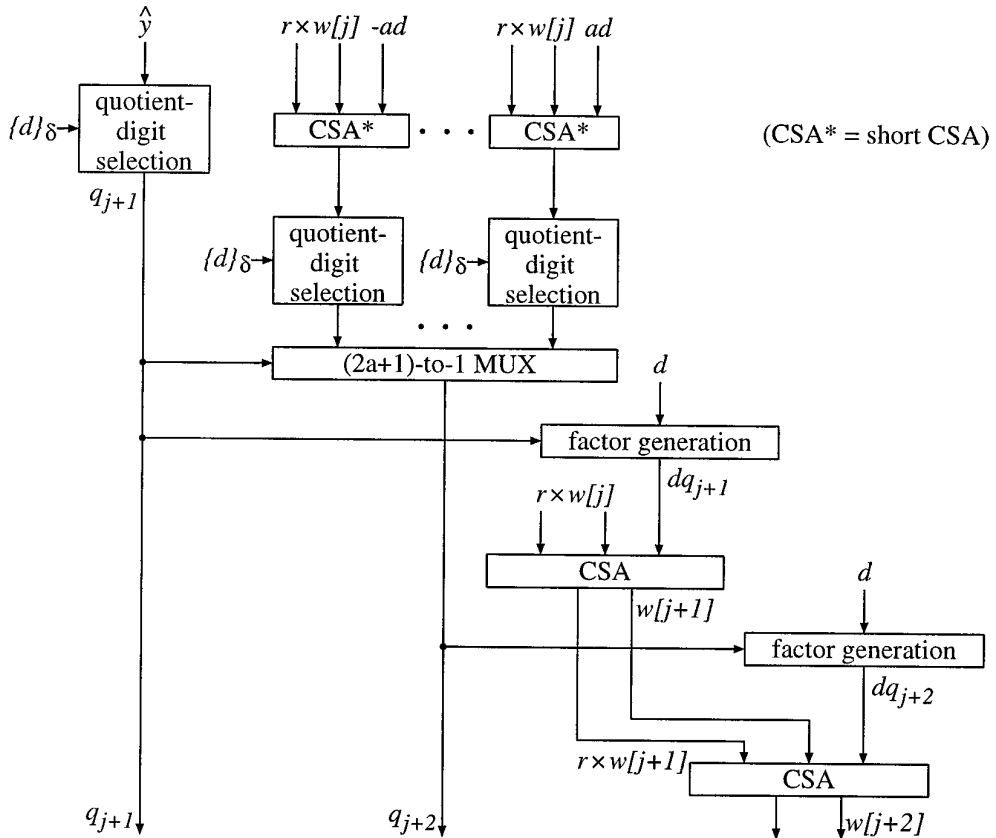


Figure 13. Overlapping quotient selection for two radix- r divider stages (adapted from Ercegovac and Lang [1994]).

ferences between microprocessor architectures, implementations, and fabrication processes. Specific examples are cited whenever available. The analysis assumes an FPU with preexisting addition and multiplication hardware into which division and square root are to be integrated.

The first choice considered is between software and hardware implementations. The discussion then moves to specific tradeoffs within multiplicative and subtractive hardware types, respectively. Last of all is a direct comparison between multiplicative and subtractive hardware implementations.

6.1 Software Versus Hardware

Implementing division and square root completely in software is obviously the

least expensive way to support these operations. Unfortunately, the cost of this hardware savings is the lowest performance of any implementation alternative. Division and square root are sufficiently important operations that software implementations should be avoided, since performance will generally suffer in comparison to even modest hardware support. Software is limited to instruction-level primitives, and cannot possibly match the speedup possible with hardware designs. Besides low cost, the only positive aspects to software implementation are simplicity, since no hardware design is required, and the flexibility to experiment with different algorithms.

Should circumstances force the implementation of square root or even divi-

sion in software, multiplicative algorithms are a much more sensible choice than subtractive methods. The fact that multiplication is the most highly-optimized operation in most floating-point units, the quadratic convergence of iterative techniques, and a more favorable set of software primitives means a much more efficient use of machine cycles and significantly higher performance overall. However, even modest hardware enhancements can deliver better performance than software alone. By moving the square root computation into hardware for the POWER2 version of the RS/6000, IBM has roughly doubled the performance of this operation from the original processor [White 1994]. An additional pitfall of multiplicative software techniques is the cost of achieving sufficient accuracy, which can actually double the latency of the operation, as in the Intel i860 [Hennessy and Patterson 1990b].

6.2 Multiplicative Hardware

While subtractive algorithms are generally implemented in wholly or partially independent units, multiplicative hardware always consists of enhancements to the existing multiplier. There is a basic area outlay required to support multiplicative division and square root, incurred by the control modifications, routing enhancements, and other changes required to transform sequences of operations into single instructions. Beyond these more or less required costs, the major implementation tradeoffs are adding hardware to support constant subtraction and shifting, choosing measures to insure last-bit accuracy, and scaling of the lookup tables.

Routing and Storage Enhancements. As suggested by Figure 2, the majority of the additions to the multiplier involve new routing paths, and routing costs are notoriously difficult to estimate. In general, the allocation of routing and storage will be dictated largely by the

specific topology of the multiplier. It may be possible to find tradeoffs between routing and storage on the one hand and execution time on the other.

The additional hardware brings a net reduction in the latency of division and square root, since it eliminates the overhead of software operations. The latency of multiplication, however, will be slightly increased since additional components have been added in series with existing ones. If floating-point multiplication is on the microprocessor's critical path, the additional routing could force a stretch in cycle time, slowing down not only all operations using the multiplier but the entire chip.

Iterative Step Support. Adding hardware support for the constant subtractions and shifts in the iteration step enhances the performance of the FPU as a whole by reducing or eliminating dependencies between the adder and multiplier. Of the two basic approaches to supporting constant subtraction, a dedicated unit is the more generally applicable; modifications to the multiplier array require detailed knowledge of its structure. The hardware needed for a dedicated subtracter should be simpler than that required for rounding, and have lower latency. Since these units are in parallel, the possibility of affecting cycle time is small.

Precision and Rounding. The nominal width of a multiplier datapath is 56 bits, which includes n , the 53 bit size of a double precision operand, along with guard, round, and sticky bits. Achieving accuracy up to the last digit for multiplicative algorithms requires either greater precision or lengthy cleanup operation in software. There are three readily available schemes for obtaining full accuracy results in hardware, and all have been implemented in general purpose processors or arithmetic coprocessors.

The Texas Instruments rounding method provides correctly rounded quotients and roots, with very little hard-

ware, and at a small performance expenditure [Darley et al. 1990; 1989]. It is also adaptable to any type of multiplier design. The multiplier datapath requires only four additional guard bits for precision and a small amount of comparison logic with the associated routing. The comparison logic only needs to compare the LSB and guard bits of the estimated numerator or radicand with the exact operand and not the full-width values, so the required logic is quite simple. Depending on the pipeline structure, a register may be required to store the relevant bits of the estimate temporarily; in addition, there is the cost of routing the lower bits of the numerator or radicand to the rounding logic. The performance expense is the latency of a single multiplication to find the estimated numerator or radicand, plus the time to execute the comparison and final rounding.

The IBM RS/6000 floating-point unit dedicates an extraordinary amount of hardware to performing a single operation, an atomic multiply-accumulate, to extremely high precision [Montoye et al. 1990]. It uses algorithms tailored to this structure to obtain proven last bit accuracy for division and square roots [Montoye et al. 1990], an approach not applicable to other floating-point topologies. The rounding scheme proposed by Kabuo et al. [1994] makes minor modifications to a floating-point multiplier of a relatively conventional type, and adds one final multiplication cycle for cleanup. While this approach requires a relatively small amount of additional hardware, its implementation is intimately linked to the particular design of the multiplier and its recoding logic.

Lookup Table Size and Convergence Rate. As mentioned in Section 4.1, the execution time of multiplicative methods and the accuracy of the initial guess are related by the quadratic convergence rate. Sarma and Matula [1993] have developed a method for deriving reciprocal tables which minimizes the relative error of the initial approxima-

tion. Table 4 shows analytically proven lower bounds on precision for optimal k -bits-in, $k + g$ -bits-out tables, where g is the number of guard digits. Note how the precision of an optimal k -bits-in, $k + g$ -bits-out lookup table is always greater than k bits; this means that the difference between the exact value and the approximation is always less than 2^{-k} .

Measuring fractional bit values may seem purely academic, but with a quadratically converging algorithm, the fractions add up to whole bits of precision over the course of several iterations. Table 5 shows the accuracy of the final reciprocal estimate as a function of the initial guess precision and number of iterations, assuming nominal quadratic convergence, for a k -bits-in, k -bits-out lookup table. The first entry in each row with 60 or more bits of accuracy is highlighted, since this is number sufficient for full accuracy using the Texas Instruments rounding method. Note how for a given number of iterations, the precision of the result varies linearly with the bits of initial precision.

For actual implementations, the designer is free to manipulate the values of k and g to obtain the required accuracy within the desired number of iterations. The cost of a k -bits-in, $k + g$ -bits-out lookup table, for either reciprocals or square root reciprocals, is

$$2^k \times (k + g),$$

which is exponential in k . By contrast, the rate of increase in accuracy is linear in k . Recall that division and square root each require their own tables, and that while a k -bits-in, $k + g$ -bits-out reciprocal lookup table reads the first k bits of the input value mantissa, the root reciprocal table requires the first $k - 1$ bits of the mantissa and the last bit of the exponent.

More recent work by Sarma and Matula [1995] proposes a way of significantly reducing lookup table area using $j + 2$ -bits-in, j -bits-out *bipartite tables*.

Table 4. Lower Bounds on the Precision of Optimal k -bits-in, $k + g$ -bits-out Reciprocal Tables for Any k

g	Precision
0	$k + .415$
1	$k + .678$
2	$k + .830$
3	$k + .912$
4	$k + .955$
5	$k + .977$

These split the input operands into two fields, feeding them to separate tables which produce positive and negative reciprocal estimates in a redundant form for multiplier recoding. The authors claim that bipartite tables require negligible latency or logic complexity over conventional techniques, while the area savings are considerable, increasing with the accuracy of the table. For example, an 8-bit-in, 8-bit out conventional table requires 256 bytes of ROM, while a 9-bit-in, 8-bit-out bipartite table only needs 120 bytes. A 16-bit-in, 16-bit-out conventional ROM is 128 Kbytes, while a 17-bit-in, 16-bit-out bipartite one is only 16 Kbytes. This method appears promising, but as of this writing has not been incorporated into any hardware divide/square root implementations.

6.3 Subtractive Hardware

Fast subtractive implementations require specialized logic unique to these operations. For this reason, they are generally implemented as wholly or partially independent units, which has the added benefit of enhancing the parallelism of floating-point computation. The typically low area requirements of the subtractive hardware makes this feasible in many designs. Nevertheless, some hardware sharing is common, generally either in the factor generation or rounding stages.

The critical path of a radix-4 SRT divider is shown in Figure 14, indicated by thick lines on routing connections and component boundaries. The compo-

nents on this path include the residual registers, quotient-digit selection function, factor generation logic, and carry-save adder. Using a carry-save adder and redundant residual reduces the delay of subtraction to an absolute minimum. The remaining components, the factor generation logic and quotient selection function, therefore become the focus of attempts to optimize division. This fact carries through to combined divide/square root units and designs with overlapping digit selection. The primary tradeoffs affecting subtractive implementations are the choice of radix and the selection of the digit set, which are interrelated, and the cost and performance implications of different levels of hardware sharing.

Choice of Radix and Digit Set. The subtractive algorithms were developed theoretically in terms of arbitrary radix r . In practice, digital implementations are limited to powers of 2. The higher the radix of an operation, the more bits per digit, and therefore the greater the number of results bits generated with every iteration. This reduces the number of iterations required to produce a result.

However, as the radix increases, so does the complexity of the selection function and factor generator. Consider three different divide/square root units with maximally redundant digit sets, as given in Table 6. The radix-2 unit has 3 result digits, while the radix-4 version has 7 and the radix-8 design has 15. Keeping the degree of redundancy equal, the number of candidate digits for selection more than doubles as the radix increases, leading to proportionally more complicated selection functions and greater delays. Eventually, the resulting increase in the cycle time of division and square root overtakes the speedup afforded by doubling the number of bits per iteration. In practice, this limit is reached for $r > 8$. Achieving practical higher radix division and square root requires advanced techniques like overlapping result-digit se-

Table 5. Final Reciprocal Approximation Precision as a Function of Initial Guess Precision and Number of Iterations for a k -bits-in, k -bits-out Lookup Table

k	Initial Guess Precision (bits)	Number of Iterations			
		1	2	3	4
4	4.415	8.830	17.660	35.320	70.640
8	8.415	16.830	33.660	67.320	134.640
12	12.415	24.830	49.660	99.320	198.640
16	16.415	32.830	65.660	131.320	262.640

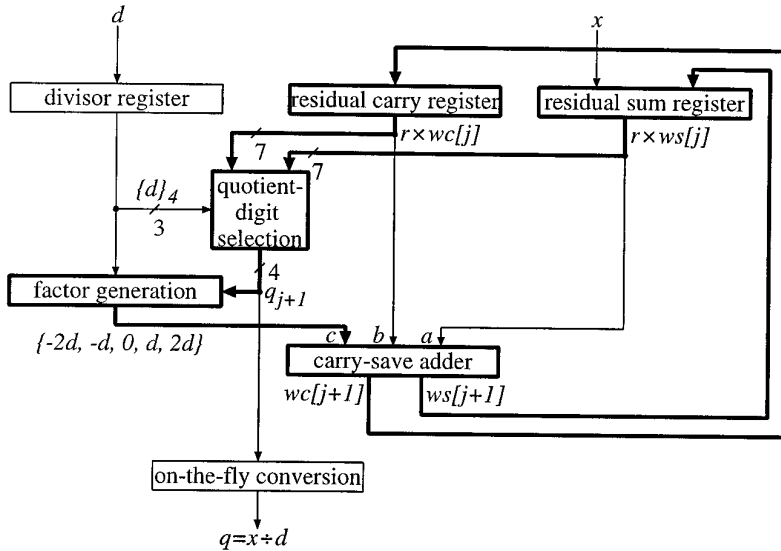


Figure 14. Radix-4 SRT divider with critical path highlighted.

lection, or the specialized very-high radix techniques described in Appendix B.

A higher degree of redundancy produces a greater amount of overlap between digit selection intervals. With a larger overlap, fewer selection constant regions are required to span the selection intervals, which means that δ , the number of divisor bits needed for the digit selection, can be reduced. These effects, illustrated in Figure 15, lead to a simplification of the result-digit selection function and an ensuing reduction in latency. The radix of the operations and degree of redundancy of the digit set affect the latency of result-digit selection in opposite ways. However, an increase in either quantity causes the number of digits and hence the cost and complexity of factor gener-

ation to increase. Digits set with $a > 2$ and/or $r > 4$ include digits which are not powers of 2, which means that factors of the divisor or partial root cannot be generated merely by shifting but require addition as well. This increases not only the amount of hardware required but also the latency of factor generation. For division, this delay is incurred only at the beginning of the operation since the divisor remains constant. In square root computation, the partial root value changes with each iteration (see Table 3), requiring the generation of new factors every cycle. Higher degrees of redundancy increase the delay of factor generation, directly counteracting the speedup of result-digit selection. Higher radices increase the delay of both operations, counteracting the decrease in the number of cycles.

Table 6. Selection of Maximally Redundant Result-Digit Sets

r	a	Digit Set	ρ
2	1	$\{-1, 0, 1\}$	1
4	3	$\{-3, -2, \dots, 2, 3\}$	1
8	7	$\{-7, -6, \dots, 6, 7\}$	1

Achieving a balance between radix, degree of redundancy, the number of cycles, and the cycle time is a complicated design problem, especially since the exact quantity of the performance and area effects is implementation-dependent. For this reason, most practical designs are based around radix-2 or minimally-redundant radix-4 stages.

Sharing Hardware Functionality. Subtractive divide/square root units are often only partially independent, sharing some hardware with the multiplier and/or adder. The divide/square root unit in the Intel Pentium, for example, shares rounding logic with the adder, while the HP PA7200 shares initial operand alignment circuitry with the multiplier. In cases like these, the shared hardware is accessed only once, either at the beginning or end of the operation. If the floating point unit can only issue and retire one operation per cycle, which is true of most designs, the performance impact is negligible.

Some designs with high-redundancy multiplier digit sets, like the Weitek 3171 arithmetic coprocessor, share factor generation logic with the multiplier. This allows division and square root to use a simplified result-digit selection function with a minimal hardware expenditure. The performance impact of sharing is negligible for division, since the logic only needs to be accessed at the beginning of the operation. Square root calculations, however, will potentially collide with multiplication at every step, since the partial root requires constant updating.

The highest degree of hardware sharing occurs in chained divide-multiply-add designs like the Mips R4400, where both the divider and multiplier send

their results, in redundant form, to the adder for conversion to standard form and rounding. While the adder processes the division results, no multiplications may complete and no additions can be issued, so the performance impact can be severe, aggravating a situation where the adder already constitutes a bottleneck. The hardware savings are slight, since on-the-fly conversion and rounding can be implemented quite economically. The Mips R4400, actually, has an even more extreme case of hardware sharing; namely, it performs square roots using the adder [Mirapuri et al. 1992]. This results in an extremely long latency, since the adder computes only one bit per iteration. In addition, it ties up the adder for the duration of the square root calculation, making it even more of a bottleneck. In summary, sharing divide/square root functionality with other functional units in the FPU is a reasonable economizing measure when the parallelism of the operations is unaffected. Any design choice which creates dependencies between units apart from the initial or final stages, or which ties up a parallel functional unit, is of questionable value.

6.4 Multiplicative Hardware vs. Subtractive Hardware

The most complex tradeoff of all is the choice between multiplicative and subtractive hardware implementations. To keep the issues as clear as possible, the discussion will be anchored by four representative implementations, listed below.

8-bit seed Goldschmidt. This is a baseline multiplicative implementation of the type in Section 4.2 and found in many actual designs. It features a modified multiplier with an 8-bit seed lookup table.

16-bit seed Goldschmidt. High-performance version of the above, enhanced with a 16-bit seed lookup table.

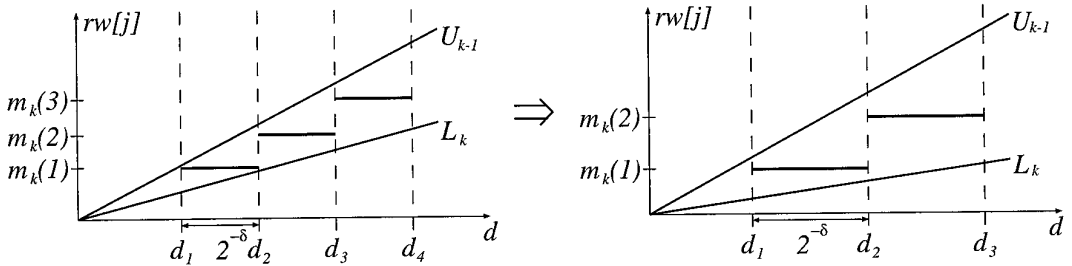


Figure 15. Effects of increasing the degree of redundancy (ρ): fewer selection constant regions (d_i) and smaller δ ($m_k(i)$ = selection constant).

radix-4 SRT. Basic implementation as in Section 5.2, subtractive equivalent of 8-bit seed Goldschmidt.

radix-16 SRT. Enhanced subtractive design featuring overlapping quotient/root selection with radix-4 stages as in Figure 13.

The selected implementations consist of two multiplicative and two subtractive members. Within each class is one basic version, as found in multiple actual FPU's, and one larger, more sophisticated, and costlier enhanced version as a possible candidate for future designs. The latency, throughput, and area properties of these four designs will be compared in order to map out the area and performance properties of multiplicative and subtractive implementations.

Latency. With their quadratic convergence, multiplicative algorithms have the potential for lower latencies than subtractive ones. In practice, there is a considerable overlap between the performance of the two classes of implementations. Table 7 shows the divide and square root performance of a selection of current microprocessors, along with the type of algorithm in the level of detail known from the literature. All of the FPU's featured have addition and multiplication latencies between 2 and four machine cycles.

Multiplicative implementations range from 9 to 20 cycles for division, 12 to 25 cycles for square root. All of the implementations shown have 8-bit seed tables.

With 16-bit seed tables, one could expect a range of 7 to 20 cycles for division, and 10 to 20 cycles for square root.

Subtractive implementations vary from 8 to 60 cycles for division and 12 to 70 cycle for square root. Surprisingly, the self-timed radix-2 SRT divider in the Hal Sparc64 (see Appendix B) actually beats out the fastest multiplicative implementation in best case performance. If the focus is restricted to radix-4 designs only, the range narrows to 15 to 39 cycles for division and 15 to 31 for square root. Assuming the balance between critical path length and cycle time can be maintained, an upgrade of these implementations to radix-16 SRT would give an estimated range of 7 to 26 cycles for division and 7 to 16 for square root. These data show that a higher convergence rate alone does not guarantee higher performance. While subtractive implementations cover a wide range of latencies, they can made competitive with multiplicative ones, or even superior in individual cases.

Throughput. Of course, latency is only one aspect of performance. It is important to consider the throughput of operations as well when making implementation decisions. As noted earlier, multiplicative divide and square root implementations are generally leveraged off of the floating-point multiplier. This means that divide, square root, and multiply operations must all share the same pipeline and are effectively serialized. In multiply-accumulate

Table 7. Recent Microprocessors and Their Divide/Square Root Algorithms and Performance (* = inferred from available information; † = not supported)

Design	Algorithm	Latency [cycles]	
		$a \div b$	$\sqrt{a}$
DEC 21164 Alpha AXP	radix-2 SRT	22–60	†
Hal Sparc64	radix-2 SRT (self-timed)	8–9	†
HP PA7200	radix-4 SRT	15	15
HP PA8000	radix-4 SRT	31	31
IBM RS/6000 POWER2	8-bit seed Newton-Raphson	16–19	25
Intel Pentium	radix-4/radix-2 SRT	39	70
MIPS R8000	8-bit seed multiplicative	20	23
MIPS R10000	SRT	18	32
PowerPC 604	SRT	31	†
PowerPC 620	SRT	18	22
Sun SuperSPARC	8-bit seed Goldschmidt	9	12
Sun UltraSPARC	radix-8 SRT	22	22

units, addition is on this list as well. Since subtractive divide and square root implementations are usually separate from other functional units, they can execute in parallel without tying up the addition and multiplication units. This allows computation to continue on other instructions while quotient or root calculation is in progress, giving the possibility of higher throughput. The degree of benefit depends on the balance of latency between functional units, and the dependencies between operations in the instruction stream.

Area. Because of differences in circuit and logic design, layout styles, and fabrication technology, comparing the area of different implementations is hard to achieve with any precision. However, theoretical estimates and data from individual cases can provide enough information to give a basis for evaluation.

Table 8 shows estimates of the relative areas of the four canonical implementations based on standard cell technology. Only the areas of circuitry devoted exclusively to divide/square root functionality are covered. The figures do not include routing costs or control logic but only the area of the datapath logic itself. Areas are displayed as a multiple of the area of the smallest implementation. The Goldschmidt implementation estimates show a range of values, where the smaller figure is

based on the use of bipartite lookup tables, and the larger one assumes a conventional unified table. The area estimates of the multiplicative alternatives should be used with particular care; because of the large tables, a slight difference in the size of a unit ROM cell can have a significant effect on overall area. Nevertheless, these figures illustrate the effects of the exponential table growth with seed accuracy, and the compression provided by bipartite table techniques.

As a supplement to the estimates, it is useful to look at some actual implementations, when available. Table 9 compares the size of the hardware required for division and square root in the Weitek 3364 and Texas Instruments 8847 arithmetic coprocessors. The figures are based on measurements of chip microphotographs [Hennessy and Patterson 1990b]. The IC's have similar die sizes and device densities, and were introduced around the same time. In short, apart from their divide/square root implementation, these two chips have a lot in common. In this instance, the multiplicative implementation is actually 30% larger than the subtractive one. Neither implementation occupies more than 7% of the FPU as a whole, and the difference is less than 1.6% of either chip's area. Although these figures represent only two particular designs, they suggest that 8-bit seed

Table 8. Relative Cost of Different Divide/Square Root Implementations

Implementation	Area Factor
8-bit seed Goldschmidt	1.0–1.2
16-bit seed Goldschmidt	22–160
radix-4 SRT	1.5
radix-16 SRT	2.2

Goldschmidt and radix-4 SRT implementations are both potentially economical, and that the area differences between them can be kept small.

Looking at all of the area data, both estimates and samples, gives an idea of the general area relationships. The 8-bit seed Goldschmidt is generally the cheapest option, but not necessarily, as the example of the Weitek and TI chips shows; the use of bipartite tables can save around 20% of the area. Radix-4 SRT implementations are of comparable area, up to around 50% larger but possibly smaller. The radix-16 SRT alternative is 50% more expensive than radix-4 SRT and 1.9 to 2.2 times larger than 8-bit seed Goldschmidt. The 16-bit seed Goldschmidt, however, shows a huge leap in area, due to exponential lookup table growth. The bipartite technique can reduce area consumption by more a factor of 7, but even the smallest implementation is approximately 15 times as big as the radix-4 SRT approach, and 10 times larger than radix-16 SRT.

7. PERFORMANCE SIMULATION OF DIVIDE/SQUARE ROOT IMPLEMENTATIONS

In the preceding section, the analysis of different implementations was based largely on static criteria, such as circuit area and the latency of individual operations. This provides an incomplete picture, since it is difficult to predict the performance of an implementation with actual programs based on static values alone. Accordingly, this section contains a series of experiments which combine representative add-multiply structures with different practical implementations of division and square root, and

Table 9. Area Comparison of Two Divide/Square Root Implementations

Algorithm	radix-4 SRT	8-bit seed Goldschmidt
Device	Weitek 3364	TI 8847
Chip Area [mm ²]	95.2	100.8
Transistor Count	165,000	180,000
Div./Sqrt. Area [mm ²]	4.8	6.3

explores the performance impact of each choice using simulation of the Givens rotation benchmark. The results provide a basis for quantitative comparison of the alternatives based on dynamic performance. A description of the Givens rotation benchmark and FPU-level simulator is followed by the experimental case studies, which are rounded off by an analysis of the results.

7.1 FPU-Level Simulation With Givens Rotations

The ultimate test of a floating-point implementation is how it performs on real programs. Givens rotation is just such an application. It is used in several common methods of solving systems of differential equations, as well as in numerous signal processing algorithms. Its sequence and combination of operations is also similar to the rotation and projection algorithms employed in 3-D graphics and solid object modeling.

Givens rotation has a high concentration of divide and square root operations, and its overall performance is particularly dependent on their implementation. In this respect it is hardly “typical”, since a great many floating-point applications use division and square root little if at all. But while it may not be typical, Givens rotation is an emphatically genuine application with important uses that demand to be supported. Its status as a divide/square root “torture test” makes it especially well-suited to bring out the strengths and weaknesses in applications.

Mathematical Description. The use of matrices and numerical analysis

techniques to model and simulate systems is ubiquitous within engineering and scientific disciplines. Many algorithms require matrices to have a certain form, such as diagonal, upper triangular, or lower triangular. Givens rotation provides a method for shaping arbitrary matrices into these and other more complicated forms required by various algorithms. Other techniques, such as Householder transforms, provide some of these capabilities, but none of them are as flexible or powerful [Golub and Van Loan 1989].

Consider two scalars a and b . Givens rotation performs the operation

$$\begin{bmatrix} c & -s \\ s & c \end{bmatrix}^T \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} r \\ 0 \end{bmatrix} \quad (4)$$

where $r = \sqrt{a^2 + b^2}$. The function in Figure 16 shows how to compute the *rotation coefficients* c and s . Let A be an m -by- n matrix representing the coefficients of a system to be modeled. In addition, let $A(i-1, j)$ and $A(i, j)$ be a and b in Equation 4, which yield particular rotation coefficients. If these values are applied to all vertical pairs of elements in rows $i-1$ and i of A in the manner of Equation 4, the result is a Givens rotation of these rows. In particular, there is a zero in $A(i, j)$ where before there was some arbitrary value. It is easy to imagine how the repeated application of Givens rotations can be used to shape a matrix into different, useful forms by the successive transformation of arbitrary values into zeros. Figure 17 shows how to triangularize an arbitrary 4-by-3 matrix by a sequence of Givens rotations. Each step requires one computation of the rotation coefficients as in Figure 16, followed by repeated applications of the rotations as per Equation 4. So although divide and square root operations are central to Givens rotation, the instruction stream is still dominated by addition and multiplication due to the large number of matrix-vector products.

There is a different method for performing Givens rotations known as the

```

function:  $[c, s] = \text{givens}(a, b)$ 
  if  $b = 0$ 
     $c = 1; s = 0$ 
  else
    if  $|b| > |a|$ 
       $\tau = a/b; s = 1/\sqrt{1 + \tau^2}; c = s\tau$ 
    else
       $\tau = b/a; c = 1/\sqrt{1 + \tau^2}; s = c\tau$ 
    end
  end
end givens

```

Figure 16. Function for computing Givens rotation coefficients.

“fast” Givens transformation. This technique uses a special matrix representation to reduce the number of multiplications by half and eliminate explicit square root operations. The fast Givens transformation was formulated in part to avoid the use of square root, and is thus a direct consequence of the traditionally poor implementations of this operation. The fast Givens transformation also suffers from the risk of overflow, and executing the tests to prevent this condition cuts into the performance advantage of using it in the first place [Golub and Van Loan 1989]. For these reasons, the standard Givens rotation method has been chosen as a benchmark for the purposes of this article.

Simulator Characteristics. The simulator models the transformation of an m -by- n matrix into upper triangular form using Givens rotations. It accepts as input the dimensions of the matrix and a function describing the structure and performance of a floating-point unit. Its output is the simulated performance of the input FPU. Note that the simulator does not actually compute the values needed for triangularization, but merely estimates the performance of different floating-point configurations in machine cycles.

In computer systems running actual workloads, floating-point performance is affected by a variety of factors outside

$$\begin{aligned}
 A = \begin{bmatrix} \times & \times & \times \\ \times & \times & \times \\ \times & \times & \times \\ \times & \times & \times \end{bmatrix} &\xrightarrow{(4,1)} \begin{bmatrix} \times & \times & \times \\ \times & \times & \times \\ \times & \times & \times \\ 0 & \times & \times \end{bmatrix} \xrightarrow{(3,1)} \begin{bmatrix} \times & \times & \times \\ \times & \times & \times \\ 0 & \times & \times \\ 0 & \times & \times \end{bmatrix} \xrightarrow{(2,1)} \begin{bmatrix} \times & \times & \times \\ 0 & \times & \times \\ 0 & \times & \times \\ 0 & \times & \times \end{bmatrix} \xrightarrow{(4,2)} \\
 \begin{bmatrix} \times & \times & \times \\ 0 & \times & \times \\ 0 & \times & \times \\ 0 & 0 & \times \end{bmatrix} &\xrightarrow{(3,2)} \begin{bmatrix} \times & \times & \times \\ 0 & \times & \times \\ 0 & 0 & \times \\ 0 & 0 & \times \end{bmatrix} \xrightarrow{(4,3)} \begin{bmatrix} \times & \times & \times \\ 0 & \times & \times \\ 0 & 0 & \times \\ 0 & 0 & 0 \end{bmatrix} = R
 \end{aligned}$$

Figure 17. Triangularization of a 4-by-3 matrix using Givens rotations.

of the FPU. For example, floating-point code is always interspersed with integer and control instructions. The quality of the nonfloating-point implementation will affect program performance whether the FPU design is close to optimal or not. This also applies to memory subsystems, which are the source of many if not most of the delays in a computer system. Such factors are beyond the control of floating-point design tradeoffs and, as such, orthogonal to this discussion and excluded from consideration or modeling by the simulator. The power of this approach is that it eliminates extraneous factors and focuses on raw floating-point performance itself. The disadvantage is that the results of the simulation will differ from the performance of code running on actual machines with FPU's similar to the ones modeled.

The simulator attempts to extract optimal performance from every FPU considered. It is designed to be as accurate as possible for a select but representative set of designs. For each configuration examined, the simulator uses a schedule derived by hand, assuming an issue rate of one floating-point instruction per cycle. For FPU's with parallel divide/square root units, the simulator attempts to overlap computation of division and square roots with other operations as much as possible.

7.2 Structure of the Experiments

The choice of add-multiply configuration largely determines the cost and

performance properties of the FPU as a whole (Section 3.2). The case studies are based on the three representative configurations, listed below, derived from actual machines in the sample of recent designs used throughout this article. Every configuration reflects a different set of design prerogatives; in each case a maximum issue rate of one operation per cycle is assumed.

Case 1. Chained add and multiply

Case 2. Independent add and multiply

Case 3. Multiply-accumulate

Each add-multiply configuration is tested with four different divide/square root implementations using the performance simulation method described in Section 7.1. These different implementations, slightly modified from Section 6.4 are listed below.

- 8-bit seed multiplicative
- 16-bit seed multiplicative
- radix-4 SRT
- radix-16 SRT

The implementations are standardized, as much as possible, across the different add-multiply configurations. Performance figures are based as closely as possible on actual implementations, using data from the FPU designs informing the add-multiply models.

The chained and independent cases have multiplicative divide/square root implementations based on Goldschmidt's algorithm. Performance esti-

mates are based on the Texas Instruments implementation [Darley 1989], taking into account the particular topology of each multiplier. The multiply-accumulate case uses a Newton-Raphson iteration, in deference to the particularities of that configuration, especially its IBM RS/6000 implementation.

Most of the FPU's which form the basis of the add-multiply configurations represented implement radix-4 division. The latency and throughput of division and square root in these designs are used in simulation, since these figures reflect the constraints of the configuration and implementation technology. The radix-16 performance figures are also based on the given radix-4 figures, derived on a case-by-case basis.

Each case study assumes that every one of the four divide/square root alternatives can be successfully incorporated into the existing add-multiply structure. In reality, some of the implementations may be precluded by external limitations. For example, the radix-16 divide/square root unit has a 20% longer cycle time than radix-4 design [Ercegovac and Lang 1994]. If the radix-4 unit only requires 83% or less of the available cycle time per iteration, then a radix-16 implementation may be feasible. If not, the required lengthening of processor cycle time will probably not be acceptable. Finally, some implementations may be proscribed by area limitations.

The benchmark used for performance evaluation is the triangularization of matrices using Givens rotations. The test data are the same for each combination of add-multiply configuration and divide/square root implementation. The selection of test matrices is based on insights into the types of problems encountered in numerical applications. Applications with square or overdetermined systems—that is, where the number of equations is greater than or equal to the number of unknowns—are far more common than ones with underdetermined systems, where there are fewer equations than unknowns. Square

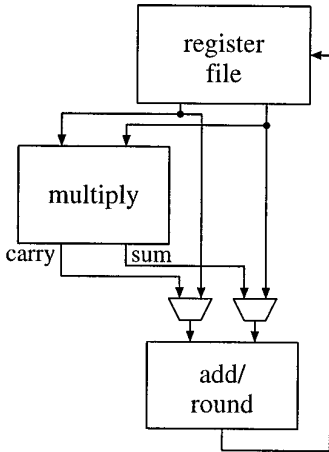
and overdetermined systems are modeled by m -by- n matrices where $m \geq n$. Also, a large proportion of the applications where the use of Givens rotations is appropriate represent smaller systems, with matrices where $n \leq 100$. With these facts in mind, the test data consists of 8 square and overdetermined matrices ranging in size from 10-by-10 to 200-by-100 elements.

7.3 Case 1: Chained Add and Multiply

The first case to be examined is a typical chained add-multiply configuration. A block diagram of this structure and the latency and throughput of addition and multiplication appear in Figure 18. This configuration is usually associated with designs where economy of area is valued over raw floating-point performance. This motivates the re-use of hardware, which makes the multiplier dependent on the adder. Typically, neither multiplication nor addition are fully pipelined, another economizing measure. The particular example in this study is inspired by the Mips R4400 microprocessor [Mirapuri et al. 1992; Simha 1993].

The latencies of division and square root for the different implementation alternatives are given in Table 10. The third implementation, which is in bold-face, is closest to the actual configuration of the Mips R4400. In the actual chip, division is performed by a radix-4 divider, while square root computation occurs in the floating-point adder using a radix-2 algorithm. Not only does this cause long square root latencies, but all operations which require the adder (i.e., all of them in this configuration) must stall while square root computation is in progress. For the sake of uniformity with other cases, the performance of division in the Mips R4400 has been applied to both operations.

The radix-16 latencies are computed as follows. Computing 53 quotient/root bits using radix-4 requires at least $\lceil 53/2 \rceil = 27$ cycles. The actual latency is 36 cycles including a 9 cycle overhead,



Operation	Latency	Throughput
add	4	3
multiply	8	4

Figure 18. Chained add-multiply configuration.

an artifact of the particular technology and FPU configuration of the Mips R4400. The estimate of radix-16 performance is based on the minimum number of cycles required, $\lceil 53/4 \rceil = 14$, plus the 9 cycle overhead from the radix-4 case, yielding a latency of 23 cycles.

Table 11 shows the improvement in execution time of the Givens rotation benchmark for each divide/square root implementation. The 8-bit seed Goldschmidt implementation is used as a performance baseline. Note the modest improvement effected by the transition from 8-bit seed to 16-bit seed Goldschmidt, as compared to the dramatic difference provided by the radix-4 and radix-16 techniques. This is a direct consequence of the enhanced parallelism of the latter designs and their ability to overlap divide and square root operations with addition and multiplication.

7.4 Case 2: Independent Add and Multiply

In the second type of add-multiply configuration, captured in Figure 19, per-

Table 10. Divide/Square Root Performance of Chained Implementations

Implementation	Latency	
	Divide	Square Root
8-bit seed Goldschmidt	35	51
16-bit seed Goldschmidt	28	40
radix-4 SRT	36	36
radix-16 SRT	23	23

formance is obviously the highest priority, and cost is less of an object. Not only are the adder and multiplier independent and fully pipelined, but their latencies are matched and only two cycles long each. In short, this FPU is built for speed. The HP PA7200 [Asprey et al. 1993; Gwennap 1994] is the inspiration for this particular example, but this general add-multiply configuration is currently the most popular in microprocessor implementations. Other chips with similar structures include the DEC 21164 [Bannon and Keller 1995], Intel Pentium [Alpert and Avnon 1993], Intel Pentium Pro [Gwennap 1995b], Mips R10000 [MIPS Technologies 1994a], Sun SuperSPARC [Sun Microsystems Computer Corp 1992], and Sun UltraSPARC [Greenly et al. 1995].

The divide and square root latencies are shown in Table 12. For radix-4 division, there is a simple but powerful optimization in effect. In the implementation technology of the HP PA7200, the cycle time of the divide/square root unit is so short compared to the latency of the multiplier array that its clock runs at twice the frequency of the rest of the system. Thus it requires only $\lceil 53/(2 \times 2) \rceil = 14$ cycles with one cycle of overhead. The radix-16 design, assuming it could be implemented with a comparable iteration delay, would therefore require $\lceil 53/(4 \times 2) \rceil + 1 = 8$ cycles. Even with these optimizations, the extremely fast multiplication makes the Goldschmidt implementations competitive in latency with the subtractive ones.

The execution time improvement figures shown in Table 13 reinforce the effects of enhanced parallelism noted

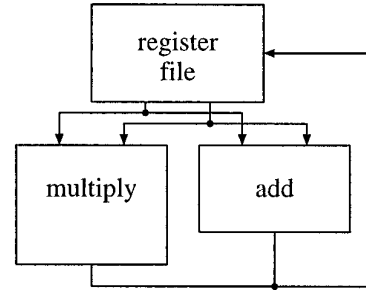
Table 11. Improvement in Execution Time [%], by Implementation, for Chained Configuration

Implementation	Max	Min	Avg
8-bit seed Goldschmidt	0.0	0.0	0.0
16-bit seed Goldschmidt	11.0	2.1	5.8
radix-4 SRT	56.2	12.9	34.2
radix-16 SRT	82.6	12.9	42.3

earlier. Although the lower multiplication latency cuts into the benefits of the radix-4 and radix-16 implementations, the performance advantages are still significant. Interestingly, the shift in balance between multiplication and addition latency from the chained configuration means that the difference between 16-bit seed Goldschmidt and 8-bit seed Goldschmidt is also smaller than before.

7.5 Case 3: Multiply-Accumulate

Like the independent configuration, the multiply-accumulate structure represents a bid for high-performance floating-point, but with a different design philosophy. Multiplication and addition are coupled, not unlike in the chained configuration, but a large amount of hardware has been devoted to bring the latency of these operations to an absolute minimum. Furthermore, addition and multiplication are performed as a single, atomic operation. The multiply-accumulate unit in this example, shown in Figure 20, is based on the IBM RS/6000 [Montoye et al. 1990] and can perform a multiply-add instruction in the same number of cycles it takes the HP PA7200 to perform just one of the operations. This configuration is capable of very high performance, particularly for the many algorithms in scientific and engineering applications which feature numerous cases of multiplication followed immediately by addition. Matrix multiplication is only one such example. Besides the IBM RS/6000 series, the Hal Sparc64 [Gwennap 1995a], HP PA8000 [Hunt 1995], and Mips R8000 [MIPS Technologies 1994b] use multiply-accumulate units.



Operation	Latency	Throughput
add	2	1
multiply	2	1

Figure 19. Independent add-multiply configuration.**Table 12.** Divide/Square Root Performance of Independent Implementations

Implementation	Latency	
	Divide	Square Root
8-bit seed Goldschmidt	9	13
16-bit seed Goldschmidt	7	10
radix-4 SRT	15	15
radix-16 SRT	8	8

The IBM RS/6000 series uses unique algorithms for the Newton-Raphson iterations (Section 4.2), due to the structure of the multiply-accumulate unit and the method used to resolve last-digit accuracy. Divide and square root latencies for the 8-bit seed Newton-Raphson implementation in Table 14 are identical to the actual processor. The 16-bit seed Newton-Raphson figures are obtained from estimates based on available information about the division and square root algorithms. The POWER2 series of processors actually has two identical floating point units, each centered on a multiply-accumulate structure. In the interest of uniformity between experiments, and to avoid the complexity of scheduling operations for two floating-point units, the simulations model the behavior of one unit in isola-

Table 13. Improvement in Execution Time [%], by Implementation, for Independent Configuration

Implementation	Max	Min	Avg
8-bit seed Goldschmidt	0.0	0.0	0.0
16-bit seed Goldschmidt	9.9	1.6	5.0
radix-4 SRT	20.9	7.2	15.2
radix-16 SRT	46.0	7.2	23.4

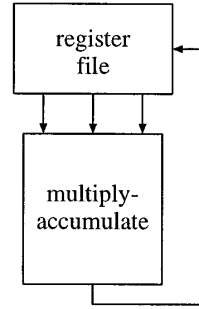
tion, as in the original POWER implementations.

When it comes to the subtractive implementations, there is a gap in the available data, since the IBM RS/6000 has only multiplicative division and square root. As an approximation, it has been assumed that the divide/square root circuits from the HP PA7200 example can be implemented alongside the IBM RS/6000 multiply-accumulate unit with the same performance values; this seems reasonable since the cycle time of the RS/6000 is actually longer than for the PA7200.

From the performance figures in Table 15, it is clear that even the multiply-accumulate configuration can profit from the parallelism of subtractive implementations. In fact, since the latency of multiplicative division and square root in cycles is slightly longer than for the independent case, the benefit is even more apparent.

7.6 Analysis

Using the data accumulated in the case studies, it is possible to draw some general conclusions about the area/performance efficiency of the different divide/square root implementations. It is important to tread lightly on the issue of comparing the performance of designs with different add-multiply configurations, for several reasons. The choice of a given configuration tends to place a design within a distinct cost/performance category. Different machines also draw the line between cycle time and cycle utility in different ways. Finally, the machines in the sample represent several different technology generations. Although the results of these



Operation	Latency	Throughput
multiply-accumulate	2	1

Figure 20. Multiply-accumulate configuration.

Table 14. Divide/Square Root Performance of Multiply-Accumulate Implementations

Implementation	Latency	
	Divide	Square Root
8-bit seed Newton-Raphson	19	25
16-bit seed Newton-Raphson	14	19
radix-4 SRT	15	15
radix-16 SRT	8	8

experiments transcend configuration boundaries, it is important to view them in light of the above qualifications.

General Observations. The single biggest factor in performance improvement, for all configurations and test matrices, is the increased parallelism of the subtractive implementations. Across the various configurations, the radix-4 SRT implementation outperforms the 8-bit seed Goldschmidt version, even with inferior per-operation latencies. It also dominates the 16-bit seed Goldschmidt on average, in spite of a consistent latency disadvantage. Even more striking is the dramatic improvement in switching from radix-4 to radix-16, compared to the relatively paltry effect of speeding up the Goldschmidt iteration with a larger seed value. Of course, not all algorithms are as readily scheduled to exploit the parallelism of subtractive

Table 15. Improvement in Execution Time [%], by Implementation, for Multiply-Accumulate Configuration

Implementation	Max	Min	Avg
8-bit seed Newton-Raphson	0.0	0.0	0.0
16-bit seed Newton-Raphson	19.7	4.9	11.6
radix-4 SRT	69.4	22.4	48.3
radix-16 SRT	125.7	22.5	68.9

implementations as the Givens rotation benchmark, but these results show the real possibilities for speedup.

Area and Performance of Specific Methods. It is enlightening to consider the performance improvement of the individual square/root divide implementations, taking into account the area investment. Table 16 reproduces the relative area estimates from Section 6 for easy reference. Table 17 shows the cumulative improvement of the benchmark execution time, across all configurations. The maximum values are more important since they generally represent the more interesting types of problems—namely small, overdetermined systems.

The 8-bit seed multiplicative implementations have, generally speaking, the lowest cost of the four alternatives for each case. However, the benchmark performance is also the worst of the implementations considered; the next slowest alternative is 1.6% to 19.7% faster.

The 16-bit seed multiplicative implementations show an enormous increase in area. This is a result of the growth of the seed lookup table—exponential in the worst case—with the number of bits of the initial guess. Unfortunately, the number of iterations required only decreases at a linear rate (Section 4), which leads to a very modest performance improvement, less than 20% in the very best case and much lower on average. This type of implementation is an extremely cost-ineffective way to perform division and square root and is

Table 16. Relative Cost of Different Divide/Square Root Implementations

Implementation	Area Factor
8-bit seed multiplicative	1.0–1.2
16-bit seed multiplicative	22–160
radix-4 SRT	1.5
radix-16 SRT	2.2

probably downright infeasible in many situations.

Radix-4 SRT divide/square root gives up to 69.4% better benchmark performance than the 8-bit seed multiplicative implementations, and never less than an 7.2% improvement. Yet the cost is no more than 50% greater. It also outperforms the 16-bit seed multiplicative implementations on average. The radix-4 implementation is arguably the most efficient balance of area and performance of the choices analyzed.

By far the swiftest of the implementation methods examined, radix-16 SRT divide/square root has a maximum performance 46.0% to 125.7% faster than corresponding 8-bit multiplicative versions, but is 2.2 times larger at worst, and at least ten times smaller than the 16-bit seed multiplicative alternatives.

Summary. On the strength of these results, subtractive implementations appear to be the soundest investment. The performance potential ranges from good to outstanding, with the ability to operate in parallel with other operations overpowering inferior latencies. Meanwhile, the sizes of entry-level configurations are modest, and enhanced performance can be achieved without an excessive investment of additional area.

While multiplicative designs include both the smallest and the largest areas in this sample, their benchmark performance is consistently inferior to the subtractive alternatives. Improving the performance of the baseline design is an expensive proposition, without much apparent profit.

Of course, this set of experiments is only a snapshot. Only three add-multiply configurations and four divide/

Table 17. Cumulative Execution Time Improvement [%] of Different Divide/Square Root Implementations

Implementation	% Improvement		
	Max	Min	Avg
8-bit seed multiplicative	0.0	0.0	0.0
16-bit seed multiplicative	19.7	1.6	7.5
radix-4 SRT	69.4	7.2	32.6
radix-16 SRT	125.7	7.2	44.8

square root options were considered out of a much larger pool of choices. Also, Givens rotation is only one application, and one which relies more heavily than most on divide and square root performance. Nevertheless, the examples were chosen to cover the range of practical implementations. And while one can expect less dramatic results from a less divide and square root intensive benchmark, these results show the performance possibilities for real applications.

8. CONCLUSIONS

Floating-point computation is becoming an increasingly high-profile feature of microcomputers. Division and square root performance in current microprocessors ranges from excellent to poor, even in chips designed for high-end applications. Designers have chosen to sacrifice these functions in favor of highly efficient addition and multiplication implementations because division and square root are perceived as relatively unimportant.

There are a number of significant, widely used applications where division and/or square root efficiency mean the difference between exceptional and poor performance. Givens rotation is an example of one such application, significant because it uses both division and square root prominently. In particularly weak implementations, divide and square root inefficiency can have an adverse effect on arithmetic performance well out of proportion with their frequency of occurrence in the code. Although a quantitative definition of ac-

ceptable divide and square root performance is elusive, treating these operations as expendable or inconsequential is a questionable judgment in light of these facts.

8.1 Guidelines for Designers

The analysis in this article is intended to help floating-point unit designers identify the divide/square root implementation type which most efficiently satisfies their performance goals and cost constraints. Rather than recommending novel methods, the focus has been on exploring the tradeoffs inherent to the established techniques used in commercial processors. These fall into the two principal classes of multiplicative and subtractive algorithms. Estimates of cost and simulations of performance, using Givens rotations as a benchmark, have been employed to evaluate the different practical alternatives.

Software implementation is the least costly alternative but also, by far, the one with the lowest possible performance, both with respect to latency and throughput. Software square root computation is uncommon in recent microprocessors, and virtually all provide some hardware support for division. Implementing either of these operations in software is highly undesirable and ought to be avoided if at all possible. If a software implementation should prove necessary, the most logical choice would be some form of multiplicative algorithm due to the more rapid convergence. However, multiplicative algorithms suffer from inherent accuracy problems, and correcting them can incur severe performance penalties—under-scoring, once again, the undesirability of software implementations. As microprocessors become larger, faster, and more elaborate, software is likely to become an increasingly less viable and attractive alternative.

It is possible to improve performance by adding hardware support for subtractive division or square root to an

existing floating point adder, but this is a poor strategy for most of the same reasons as for software implementations. A more effective method is to add hardware enhancements to the floating-point multiplier in support of the Goldschmidt's algorithm, the Newton-Raphson method or similar algorithms. This involves, as a practical minimum, extra routing and a lookup table; hardware support for constant subtraction and last digit accuracy is also highly recommended. This approach improves the latency of operations above the level of software performance, but forces the multiplier to perform double or triple duty as a divide/square root unit. Implementing 8-bit seed multiplicative algorithms requires only a modest hardware investment. Significantly improving the performance of multiplicative implementations requires an increase in the seed lookup table size. This produces only modest performance gains with a substantial increase in cost, perhaps as much as by a factor of 160, which is generally not a practical implementation option. In general, multiplicative methods should only be used if transistor budgets or architectural constraints prevent the implementation of a separate subtractive unit.

The maximum division and square root performance can be realized by including separate, subtractive hardware executing in parallel with the other operations. In the Givens rotation benchmark simulations, radix-4 SRT implementations outperform both 8-bit seed and 16-bit seed multiplicative units in the majority of cases, in spite of longer operation latencies. This performance advantage, even for small problems with lots of dependencies, is due to the parallel execution of division and square root. Furthermore, the cost is competitive with the area required for 8-bit seed multiplicative operations. The performance lead becomes even more decisive if the technology is upgraded to a higher radix, such as a radix-16 SRT unit with overlapping quotient selection, which may be imple-

mented in around twice the area required for an 8-bit seed multiplicative unit. The greatest challenge to implementing higher-radix designs is matching operations with processor cycle time. Designers with lots of available area for division and square root and the need for very low latencies may wish to consider some of the more radical implementation styles, like self-timed or very high radix methods (Appendix B).

8.2 Future Trends

Looking at the bigger picture, current trends in microprocessor implementation include ever larger transistor budgets and successively higher levels of parallelism. Designers are increasingly less likely to worry about conserving area than to puzzle over how to use available space efficiently. The latest decoupled superscalar processors issue up to four instructions per cycle, and are capable of efficiently scheduling a large number of functional units. Subtractive methods, with their parallel operation, are in a better position to exploit this situation than multiplicative techniques, which serialize multiplication, division, and square root computation. As the need to conserve area and devices becomes less urgent, one of the primary motivations for multiplicative methods begins to recede, leaving latency as the primary incentive.

In addition, multiplicative implementations are always more or less intimately linked to the design of the floating-point multiplier, possibly compromising its performance. Subtractive techniques decouple division and square root from multiplication and provide the possibility of independently upgrading the implementations. Though multiplication may not be an ideal match for division and square root, the combination of multiplication and addition has strong arguments in its favor. Implementations like the HP PA8000, which puts multiply-accumulate circuits in parallel with SRT divide/square root

units could become increasingly common.

Finally, as microprocessor cycle times persist in their decline, the pressure for improved performance increases for all operations. Floating-point and addition and multiplication continue to meet the challenge, and division and square root will have to improve, both in latency and throughput, to keep up with other operations and avoid being a drag on the FPU as a whole. Recent microprocessors feature some of the most elaborate implementations to date, such as the Sun UltraSPARC's radix-8 divide/square root unit, or the self-timed, heavily pipelined divider in the Hal Sparc64. In all likelihood, these are just the first examples of a trend towards increasingly sophisticated floating-point divide and square root hardware.

A. THE INTEL PENTIUM FDIV BUG

The error in the division portion of the floating-point unit of the Pentium microprocessor drew a lot of media attention late in 1994. An interesting artifact of the error was that it exposed the internals of the Pentium design to public scrutiny. Several individuals managed to reverse-engineer the implementation as a result of the design error. If the division unit had worked correctly, it would have been impossible to determine the nature of the implementation.

Intel uses a radix-4 SRT implementation for division in the Pentium. This is a change from the division unit in the 486 which used radix-2 representation. Details of the Pentium implementation and the design error are available in a white paper published by Intel [Sharangpani and Barton 1994]. The error resulted from incorrect entries in the lookup table used to implement the quotient selection in the SRT algorithm (see Section 5). Five entries that should have had the value 2 had the value 0. These entries are on the border between the top of the table and the don't care region. According to Intel, the error occurred when the five entries were mis-

takenly omitted from the input script to the PLA (programmable logic array) used to generate the circuit layout for the table.

The error manifests itself only for a small minority of operands, but can be considerable when it appears. For example, one would expect the result of

$$r = x - (x/y) \times y$$

to be zero. Calculating r with values $x = 4195835$ and $y = 3145727$ on a faulty Intel Pentium results in $r = 256$. The division x/y in this case is accurate to only 14 bits rather than 53, i.e., worse than single precision for a double precision computation, and is an example of the worst-case error [Moler 1995]. The only inputs affected are those which at some time in the course of computation encounter the missing entries. Finding a narrow bound on these values is more difficult than one might first imagine, but Coe and Tang [1995] have proven that all operand pairs susceptible to the error must have a divisor of the form $0.xxxx111111$.

The bug affects only division and not square root, which does not use the table. In fact, this is one of the most intriguing aspects of this case. Ironically, the fateful quotient selection table actually appears over-designed, with higher resolution than necessary for radix-4 SRT division. Meanwhile, the Pentium implements square root using a radix-2 SRT algorithm, instead of implementing radix-4 square root as well; this aspect of the floating-point unit design does not appear to have changed dramatically from the Intel 486 implementation. In short, the divide/square root implementation itself, as well as the actual division bug, seem to indicate an unfocused and fragmented design process.

This public relations and financial fiasco (Intel wrote off \$475 million in costs to correct the error) points out the dangers in changing a design without a thorough understanding of the implementation and sufficient care in insur-

ing correctness. It also shows that divide and square root implementations are an active area of redesign in modern high-performance microprocessors.

B. ADVANCED SUBTRACTIVE ALGORITHMS AND IMPLEMENTATIONS

There have been many proposals for improving the speed of subtractive divide and square root operations beyond conventional SRT. These involve novel algorithms, implementation-level refinements, or some combination of the two. Out of these numerous options, two general types, *self-timing* and *very high radix* methods appear to have the potential for significant impact on microprocessor FPU design. At this writing, the commercial implementation of these methods has been limited to one multi-chip RISC CPU with a self-timed divider and one arithmetic coprocessor with very high radix division. This is indicative not only of the rarity of these methods, but a reflection of their above-average area requirements. Nevertheless, increased miniaturization and the push towards ever-lower latencies for division and square root might eventually bring these techniques into the mainstream.

The purpose of this section is not to explain these methods in detail, but rather to provide a brief, high-level overview with references for additional reading. Oberman and Flynn [1994a] have written a very thorough and readable survey which covers these and other advanced division techniques, and is a good starting point for further investigation.

B.1. Self-Timed Division/Square Root

The idea of self-timing goes beyond divide and square root implementation. Conventional synchronous circuits consist of combinational logic blocks separated by clocked latches. Dividing computation between blocks so as to achieve maximum utilization of machine cycles can be challenging, especially for complex, long-la-

tency operations like division and square root. Self-timed designs use dynamic logic to dispose of latches, and asynchronous design methods to let circuits run at their own speed independent of any global clock, making the most efficient use of available time.

Williams [1991] describes a divider implementation which employs not only self-timing but extensive pipelining, speculative execution, and detection of early completion. It consists of a self-timed ring of 5 radix-2 SRT stages, each stage containing one full-width carry-propagate adder (CPA), two truncated CPA's, and two truncated CSA's. One consequence of the divider's asynchronous operation is a data-dependent execution time; in 1.2 μm CMOS technology, it has a latency of 45 ns to 160 ns for double-precision operands. A variant of this divider is featured in the Hal Sparc64 multichip microprocessor, which produces quotients in 8 or 9 machine cycles. Matsubara et al. [1995] have simulated but not implemented an extension of William's design to include square root computation. Their projected worst-case execution time for either operation is only 30 ns when fabricated in 0.3 μm CMOS technology intended for 200 MHz-clocked microprocessors, or 6 machine cycles.

Self-timed designs can operate 2 to 3.5 times faster on a per-operation basis than conventional radix-4 SRT implementations in similar silicon technologies. They take the standard SRT policy of independent functional units to an even higher-level of specialization, and can therefore reap the same benefits of parallel operation. Area estimates indicate that self-timed divide/square root circuits run from 1.8 to 2.5 times the size of technology-equivalent radix-4 SRT devices, which is less than or equal to the improvement in performance. Possible problems include the scheduling of data-dependent operations, and the difficulty of testing an asynchronous functional unit in a synchronous processor, but these are not insurmountable challenges. Given their attractive fea-

tures, it seems reasonable to anticipate a more widespread implementation of self-timed divide and square root units in the future.

B.2. Very High Radix Methods

While the self-timed designs described above represent fairly radical, aggressive implementations, the algorithmic basis remains relatively conventional. Very high radix methods, by comparison, make significant departures from SRT orthodoxy. Recall that the primary barrier to higher radix division in conventional implementations is the increasing complexity of quotient selection. Even with various forms of staging and hardware replication, a limit is reached where the critical path simply becomes too long for practical implementation, or the area grows out of reasonable proportions.

Very high radix methods operate with radices from 2^{10} to 2^{18} or even higher. This is achieved by shifting complexity from quotient selection to the production of divisor factors. There are a variety of formulations by Wong and Flynn [1992], Briggs and Matula [1993], and Ercegovac et al. [1993], among others. Although they differ in the details, these techniques have in common the simplification of quotient selection to a multiplication or rounding operation, the use of multiplication for factor formation, and lookup tables for initial reciprocal approximation. The sole commercial implementation to date is in the Cyrix 83D87 arithmetic coprocessor [Briggs and Matula 1993].

The primary advantage of very high radix division is the possibility of very low latencies, claimed to be the smallest of any known method. For example, compared to a self-timed radix-16 driver with overlapped radix-4 stages in the same technology, estimated performance ranges from competitive for a simple implementation, to 85% faster for a more complex one. The major disadvantage is the required area; efficient implementations require one or several

reduced-precision multipliers in addition to a lookup table and specialized logic. Complete implementations range in size from roughly half the area of a full multiplier array to several times larger [Ercegovac et al. 1993]. This is an unusually large area to devote to division, which raises another issue: most very high radix algorithms are for division only, not square root. Lang and Montuschi [1995] have shown how to combine square root computation with the method of Ercegovac et al. [1993], but other examples are hard to find.

Very high radix methods are in their infancy compared to more conventional subtractive and multiplicative techniques. With further refinement, and with increasingly generous area allocations for division and square root, they may well find a place in microprocessor FPU's.

ACKNOWLEDGMENTS

We would like to thank Adam Bojanczyk, Harold Stone, Earl Swartzlander, and William Kahan for reviewing earlier versions of this paper and providing valuable comments. We are also grateful to the journal referees for suggesting improvements to the article.

REFERENCES

- ALPERT, D. AND AVNON, D. 1993. Architecture of the Pentium microprocessor. *IEEE Micro* (June) 11–21.
- ANDERSON, S. F. ET AL. 1967. The IBM System/360 Model 91: Floating-point execution unit. *IBM J. Res.* (Jan.) 34–53.
- ASPREY, T. ET AL. 1993. Performance features of the PA7100 microprocessor. *IEEE Micro* (June) 22–35.
- ATKINS, D. E. 1968. Higher-radix division using estimates of the divisor and partial remainders. *IEEE Trans. Comput.* (Oct.) 925–934.
- BANNON, P. AND KELLER, J. 1995. Internal architecture of Alpha 21164 microprocessor. In *Digest of Papers: COMPCON Spring 1995*, IEEE, Feb. 79–87.
- BECKER, M. C. ET AL. 1993. The PowerPC 601 microprocessor. *IEEE Micro* (Oct.) 54–68.
- BERRY, M., CHEN, D., KOSS, P., AND KUCK, D. 1988. The Perfect Club benchmarks: Effective performance evaluation of supercomputers. CSRD Rep. No. 827, Center for Supercom-

- puting Research and Development, University of Illinois at Urbana-Champaign, Nov.
- BERRY, M., CYBENKO, G., AND LARSON, J. 1991. Scientific benchmark characterizations. *Parallel Comput.* (Dec.) 1173–1194.
- BRIGGS, W. S. AND MATULA, D. W. 1993. A 17×69 multiply and add unit with redundant binary feedback and single cycle latency. In *Proceedings of the 11th IEEE Symposium on Computer Arithmetic*, June 163–170.
- BURGESS, B. ET AL. 1994. The PowerPC 603 RISC microprocessor. *Commun. ACM* (June) 34–42.
- CASE, B. 1995. SPEC95 retires SPEC92. *Microprocessor Rep.* (Aug.) 11–14.
- COE, T. AND TANG, P. T. P. 1995. It takes six ones to reach a flaw. In *Proceedings of the 12th IEEE Symposium on Computer Arithmetic*, IEEE, July 140–146.
- CONTE, T. M. 1993. Architectural resource requirements of contemporary benchmarks: A wish list. In *Proceedings of the 26th Annual Hawaii International Conference on System Sciences*, IEEE, Jan. 517–529.
- DARLEY, H. M. ET AL. 1989. Floating-point/integer processor with divide and square root functions. U.S. Patent 4,878,190, Oct.
- DARLEY, M. ET AL. 1990. The TMS390C602A floating-point coprocessor for Sparc systems. *IEEE Micro*, (June) 36–47.
- DIXIT, K. M. 1991. The SPEC benchmarks. *Parallel Computing*, (Dec.) 1195–1209.
- ERCEGOVAC, M. D. AND LANG, T. 1994. *Division and Square Root: Digit Recurrence Algorithms and Implementations*. Kluwer Academic Publishers; Norwell, MA.
- ERCEGOVAC, M. D., LANG, T., AND MONTUSCHI, P. 1993. Very high radix division with selection by rounding and prescaling. In *Proceedings of the 11th IEEE Symposium on Computer Arithmetic*, IEEE, June 112–119.
- FOWLER, D. L. AND SMITH, J. E. 1989. An accurate, high speed implementation of division by reciprocal approximation. In *Proceedings of the 9th IEEE Symposium on Computer Arithmetic*, Sept. 60–67.
- FRANTZESKAKIS, E. N. AND LIU, K. J. R. 1994. A class of square root and division free algorithms and architectures for QRD-based adaptive signal processing. *IRE Trans. Signal Process.* (Sept.) 2455–2469.
- GOLUB, G. H. AND VAN LOAN, C. F. 1989. *Matrix Computations*. The Johns Hopkins University Press, Baltimore, Second Ed.
- GREENLY, D. ET AL. 1995. UltraSPARC^(tm), The next generation superscalar 64-bit SPARC. In *Digest of Papers: COMPCON Spring 1995*, IEEE (Feb.) 442–451.
- GROSS, T. 1985. Software implementation of floating-point arithmetic on a reduced-instruction-set processor. *J. Parallel Distrib. Comput.* 2, 362–375.
- GWENNAP, L. 1994. PA-7200 enables inexpensive MP systems: HP's next-generation PA-RISC also contains unique "assist" cache. *Microprocessor Rep.* (Jan.).
- GWENNAP, L. 1995a. Hal reveals multichip SPARC processor: High-performance CPU for Hal systems only—no merchant sales. *Microprocessor Rep.* 9, 3 (Mar.) 1,6–11.
- GWENNAP, L. 1995b. Intel's P6 uses decoupled superscalar design: Next generation of x86 integrates L2 cache in package with CPU. *Microprocessor Rep.* 9, 2 (Feb.) 9–15.
- HENNESSY, J. L. AND PATTERSON, D. A. 1990a. *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann Publishers, San Mateo, CA.
- HENNESSY, J. L. AND PATTERSON, D. A. 1990b. *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann Publishers, San Mateo, CA. Appendix A: Computer Arithmetic by David Goldberg.
- HUNT, D. 1995. Advanced performance features of the 64-bit PA-8000. In *Digest of Papers: COMPCON Spring 1995*, IEEE, Feb. 123–128.
- IEEE STANDARD FOR BINARY FLOATING-POINT ARITHMETIC. 1985. New York ANSI/IEEE Std. 754—1985, Aug.
- JULIUSSEN, E. 1994. Which low-end workstation? *IEEE Spectrum*, (Apr.) 51–59.
- KABUO, H. ET AL. 1994. Accurate rounding scheme for the Newton-Raphson method using redundant binary representation. *IEEE Trans. Comput.* (Jan.) 43–50.
- KAHAN, W. 1994. Using MathCAD 3.1 on a Mac, Aug.
- KOHN, L. AND MARGULIS, N. 1989. Introducing the Intel i860 64-bit microprocessor. *IEEE Micro*, (Aug.) 15–30.
- LANG, T. AND MONTUSCHI, P. 1995. Very-high radix combined division and square root with prescaling and selection by rounding. In *Proceedings of the 12th IEEE Symposium on Computer Arithmetic*, IEEE, July 124–131.
- LU, P. Y. AND DAWALLU, K. 1989. A VLSI module for IEEE floating-point multiplication/division/square root. In *Proceedings of the IEEE International Conference on Computer Design: VLSI in Computers and Processors*, 366–368.
- MARKSTEIN, P. W. 1990. Computation of elementary functions on the IBM RISC System/6000 processor. *IBM J. Res. Develop.* (Jan.) 111–119.
- MATSUBARA, G., ET AL. 1995. 30-ns 55-b shared radix-2 division and square root using a self-timed circuit. In *Proceedings of the 12th IEEE Symposium on Computer Arithmetic*, IEEE, July 98–105.
- MCLELLAN, E. 1993. The Alpha AXP architecture.

- ture and 21064 processor. *IEEE Micro*, (June) 36–47.
- MCQUILLAN, S. E. AND MCCANNY, J. V. 1991. A VLSI architecture for multiplication, division, and square root. In *Proceedings of the 1991 International Conference on Acoustics, Speech and Signal Processing*, IEEE, May 1205–1208.
- MCQUILLAN, S. E., MCCANNY, J. V., AND HAMILL, R. 1993. New algorithms and VLSI architectures for SRT division and square root. In *Proceedings of the 11th IEEE Symposium on Computer Arithmetic*, IEEE, June 80–86.
- MIPS TECHNOLOGIES, INC. 1994a. Mountain View, CA. *R10000 Microprocessor: Product Overview*, Oct.
- MIPS TECHNOLOGIES, INC. 1994b. Mountain View, CA. *R8000 Microprocessor Chip Set: Product Overview*, Aug.
- MIRAPURI, S., WOODACRE, M., AND VASSEGGHI, N. 1992. The Mips R4000 processor. *IEEE Micro* Apr. 10–22.
- MISRA, M. 1990. *IBM RISC System/6000 Technology*. IBM.
- MOLER, C. 1995. A tale of two numbers. *SIAM News* 28, 1 Jan. 1,16.
- MONTOYE, R. K., HOKENEK, E., AND RUNYON, S. L. 1990. Design of the IBM RISC System/6000 floating-point execution unit. *IBM J. Res. Develop.* (Jan.) 59–70.
- OBERMAN, S. F. AND FLYNN, M. J. 1994. An analysis of division algorithms and implementations. Tech. Rep. CSL-TR-95-675, Stanford Univ. Departments of Electrical Engineering and Computer Science, Stanford, CA, Dec.
- OBERMAN, S. F. AND FLYNN, M. J. 1994. Design issues in floating-point division. Tech. Rep. CSL-TR-94-647, Stanford University Departments of Electrical Engineering and Computer Science, Stanford, CA, Dec.
- PENG, V., SAMUDRALA, S., AND GAVRIELOV, M. 1987. On the implementation of shifters, multipliers, and dividers in VLSI floating point units. In *Proceedings of the 8th IEEE Symposium on Computer Arithmetic*, IEEE, May 95–102.
- SARMA, D. D. AND MATULA, D. W. 1993. Measuring the accuracy of ROM reciprocal tables. In *Proceedings of the 11th IEEE Symposium on Computer Arithmetic*, (June) 95–102.
- SARMA, D. D. AND MATULA, D. W. 1995. Faithful bipartite ROM reciprocal tables. In *Proceedings of the 12th IEEE Symposium on Computer Arithmetic*, IEEE (July) 17–28.
- SCOTT, N. R. 1985. *Computer Number Systems and Arithmetic*. Prentice Hall, Englewood Cliffs, NJ.
- SHARANGPANI, H. P. AND BARTON, M. L. 1994. Statistical analysis of floating point flaw in the Pentium™ processor. Tech. Rep., Intel Corporation, Nov.
- SIMHA, S. 1993. *R4400 Microprocessor: Product Information*. MIPS Technologies, Inc., Mountain View, CA. Sept.
- SONG, S. P. ET AL. 1994. The PowerPC 604 RISC microprocessor. *IEEE Micro* (Oct.) 8–17.
- STEARNS, C. C. 1989. Subtractive floating-point division and square root for VLSI DSP. In *European Conference on Circuit Theory and Design*. Sept. 405–409.
- SUN MICROSYSTEMS COMPUTER CORPORATION. 1992. Mountain View, CA. *The SuperSPARC™ Microprocessor*, May.
- TAYLOR, G. S. 1985. Radix 16 SRT dividers with overlapped quotient selection stages. In *Proceedings of the 7th IEEE Symposium on Computer Arithmetic*, IEEE, June 64–71.
- WASER, S. AND FLYNN, M. J. 1982. *An Introduction to Arithmetic for Digital System Designers*. Holt, Rinehart and Winston, New York.
- WEICKER, R. P. 1991. A detailed look at some popular benchmarks. *Parallel Computing*, (Dec.) 1153–1172.
- WHITE, S. W., ET AL. 1993. How does processor MHz relate to end-user performance? Part 1: Pipelines and functional units. *IEEE Micro*, (Aug.) 8–16.
- WHITE, S. W. 1994. POWER2: Architecture and performance. In *Digest of Papers: COMPCON Spring 1994*, IEEE, Feb. 384–388.
- WILLIAMS, T. E. 1991. A zero-overhead self-timed 160-ns 54-b CMOS divider. *IEEE J. Solid-State Circuits*, Nov. 1651–1661.
- WONG, D. AND FLYNN, M. 1992. Fast division using accurate quotient approximations to reduce the number of iterations. *IEEE Trans. Comput.* (Aug.) 981–995.

Received February 1995; revised February 1996; accepted June 1996